

LLM Training Strategies

(Pre-training + IFT+ PEFT)

ELL8299 · COL8383

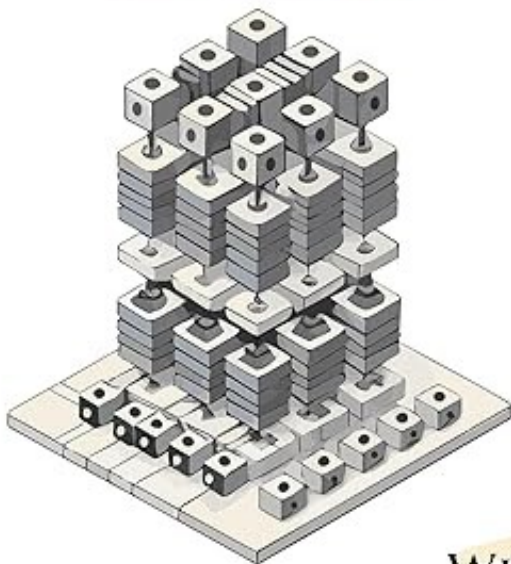
Tanmoy Chakraborty
Professor, IIT Delhi
<https://tanmoychak.com/>



INTRODUCTION TO LARGE LANGUAGE MODELS

Generative AI for Text

Tanmoy Chakraborty



WILEY

Chapter 7



LLMs | Neural Language Models:
LSTM and GRU | Lec 5.2

LCS2



LLMs | Neural Language Models:
Seq2Seq and Attention | Lec 5.3

LCS2



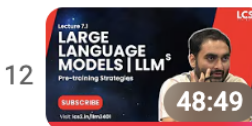
LLMs | Introduction to
Transformer: Self & Multi-Head...

LCS2



LLMs | Intro to Transformer:
Positional Encoding and Layer...

LCS2



LLMs | Pre-training Strategies |
ELMo & BERT | Lec 7

LCS2

Intro to LLMs – 2024



Lec 05: Part A | LLM Pre-training

821 views • 10 months ago



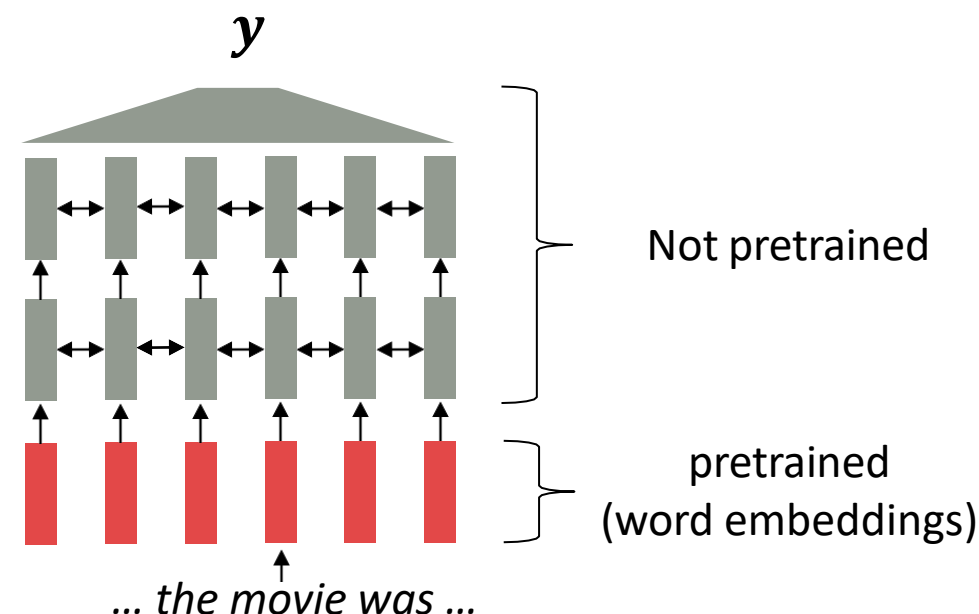
Lec 05: Part B | LLM Instruction Tuning

843 views • 10 months ago

Adv LLMs – 2025

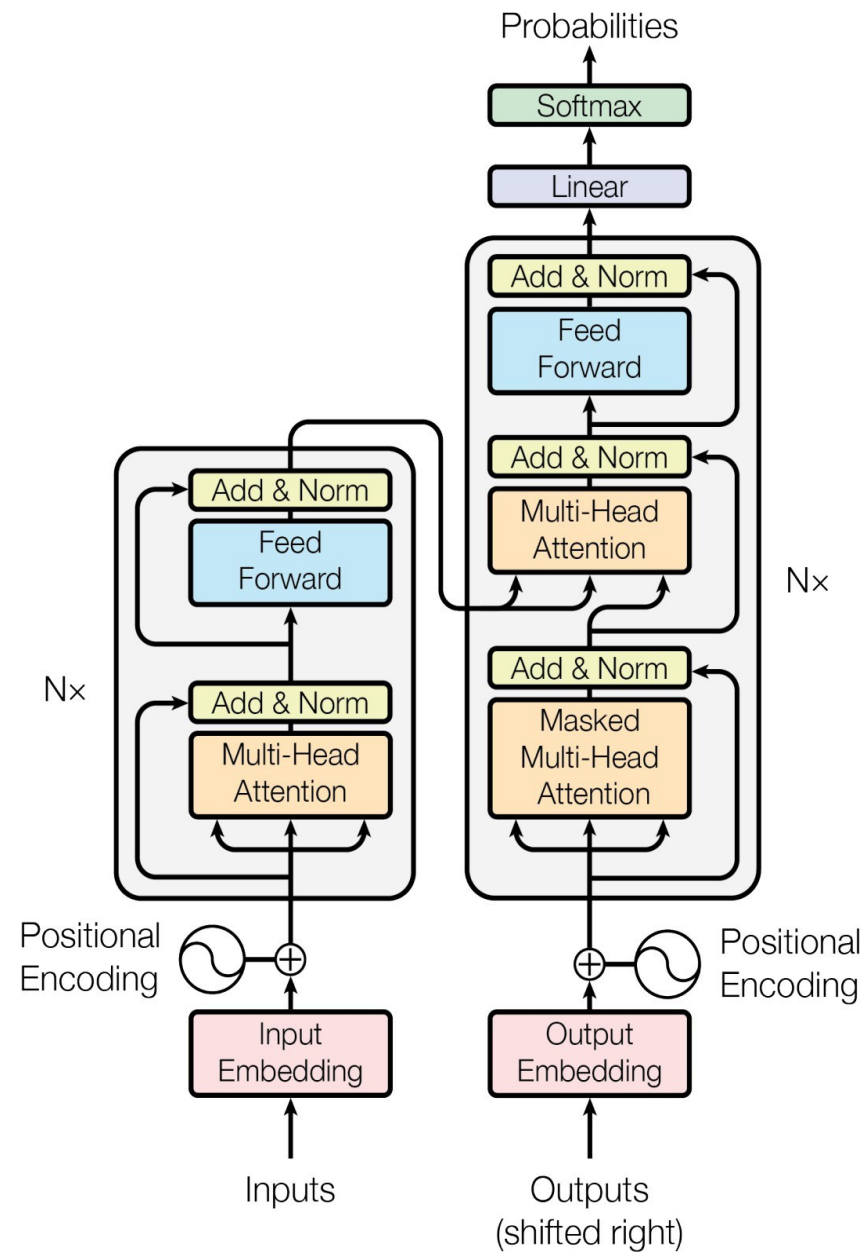
Where We Were: Pre-trained Word Vectors

- Start with pretrained word embeddings (no context!)
- Learn how to incorporate context in an LSTM or Transformer while training on the task.
- The training data we have for our **downstream task** (like question answering) must be sufficient to teach all contextual aspects of language.
- Most of the parameters in our network **are randomly initialized!**



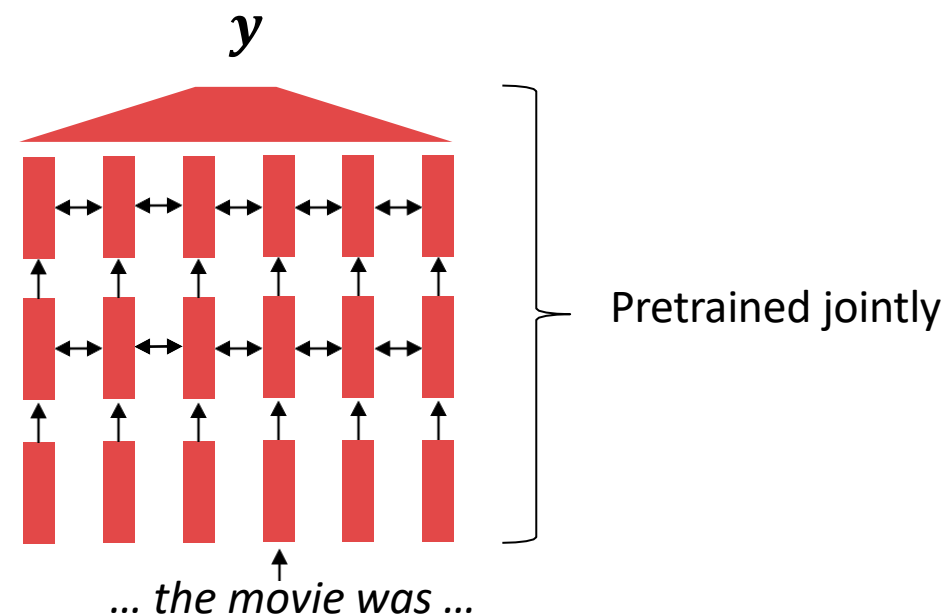
Recall: Encoder-Decoder Architecture

- Standard Transformer Architecture
- Decoder attends back to the input. But the input doesn't change, so this just needs to be encoded once.



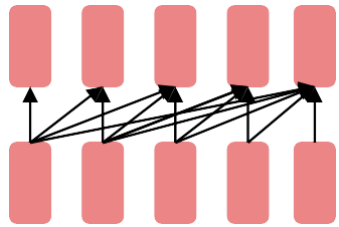
Pre-trained Word Vectors -> Pre-trained Models

- All (or almost all) parameters in NLP networks are initialized via **pretraining**.
- Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts.
- This has been exceptionally effective at building strong:
 - **representations of language**
 - **parameter initializations** for strong NLP models.
 - **Probability distributions** over language that we can sample from



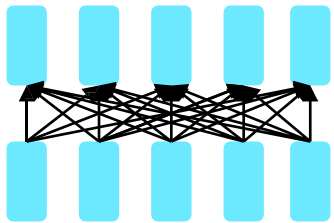
Pretraining for Three Types of Architectures

The neural architecture influences the type of pretraining, and natural use cases.



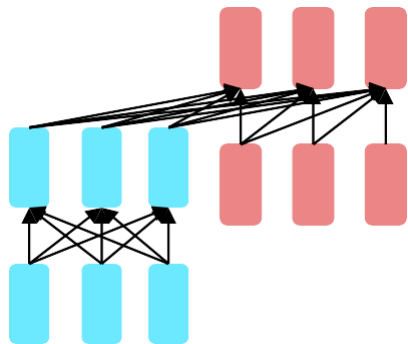
Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words



Encoders

- Gets bidirectional context – can condition on future!
- How do we pretrain them?



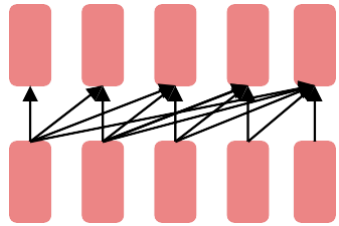
**Encoder-
Decoders**

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Pretraining for Three Types of Architectures

The neural architecture influences the type of pretraining, and natural use cases.



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words



Auto-regressive Language Models

$$P(X) = \prod_{i=1}^I P(x_i \mid x_1, \dots, x_{i-1})$$


The diagram illustrates the components of the probability formula. A red horizontal line segment is positioned under the term x_i in the numerator, with a red arrow pointing from the text "Next Token" below it to this segment. A blue horizontal line segment is positioned under the denominator $x_1, \dots, x_{i-1}$, with a blue arrow pointing from the text "Context" below it to this segment.



Next Token Prediction

- This is essentially **classification**!
 - We can think of neural language models as neural classifiers. They classify prefix of a text into $|V|$ classes, where the classes are vocabulary tokens.

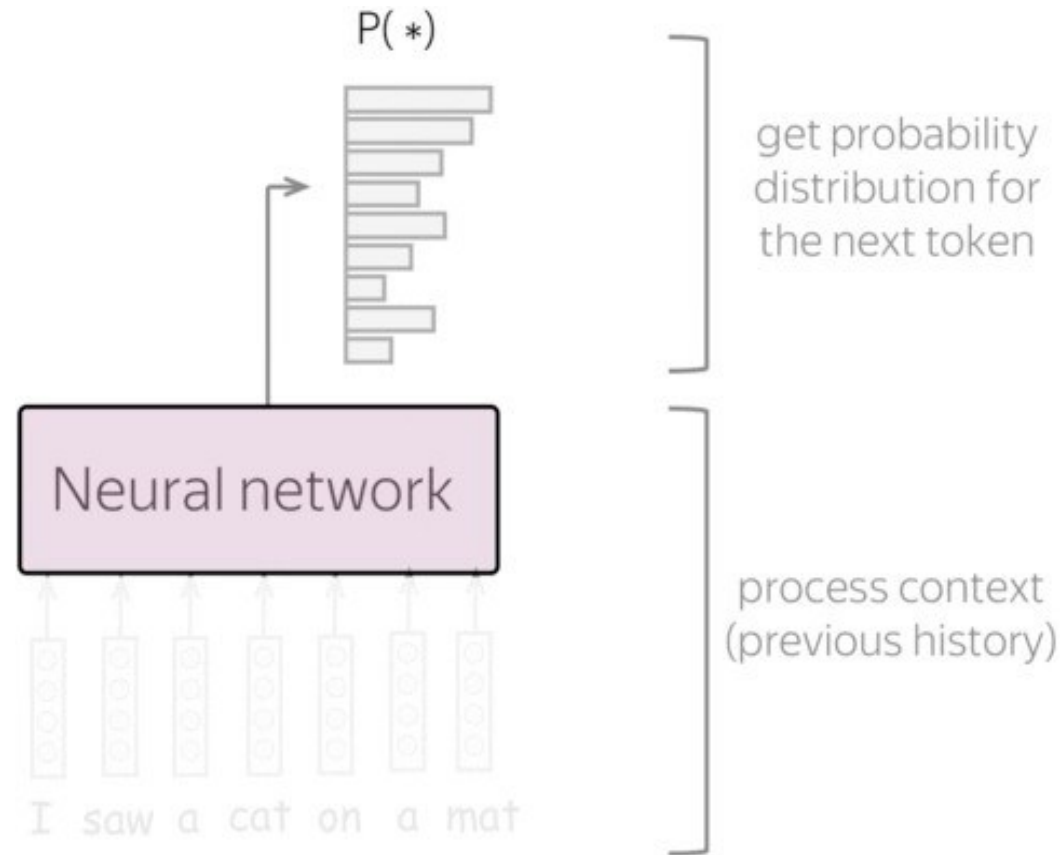
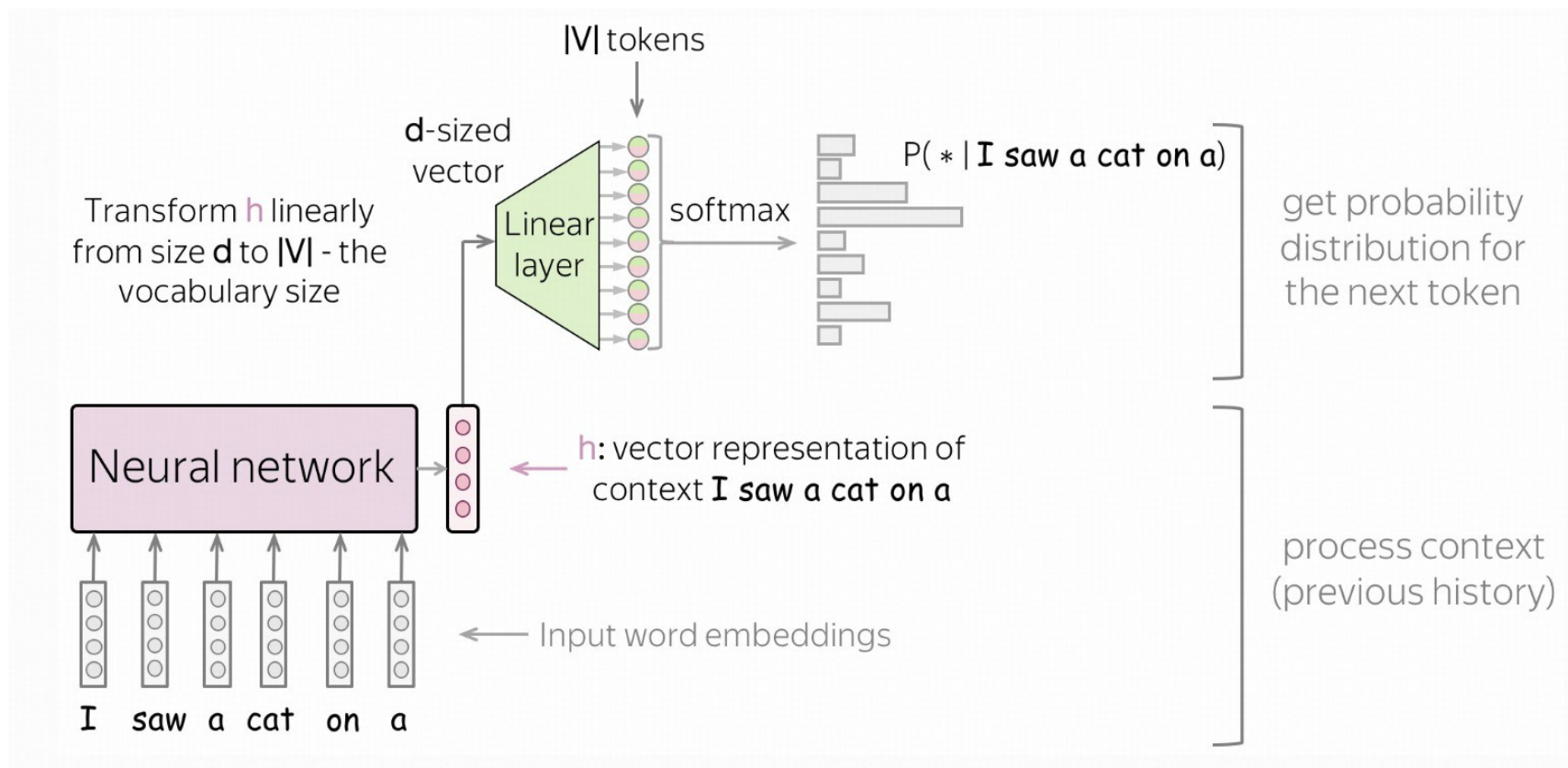


Image Credit: https://lena-voita.github.io/nlp_course/language_modeling.html



Next Token Prediction



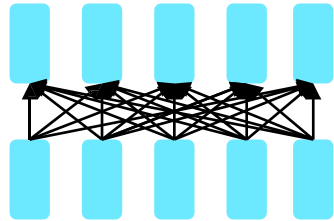
- Feed word embedding for previous (context) words into a network.
- Get vector representation of context from the network.
- From this vector representation, predict a probability distribution for the next token.

Image Credit: https://lena-voita.github.io/nlp_course/language_modeling.html



Pretraining for Three Types of Architectures

The neural architecture influences the type of pretraining, and natural use cases.



Encoders

- Gets bidirectional context – can condition on future!
- How do we pretrain them?



BERT: Bidirectional Encoder Representations from Transformers

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI Language

`{jacobdevlin, mingweichang, kentonl, kristout}@google.com`

Slides are adopted from Jacob Devlin



Masked Language Modelling

- Mask out $k\%$ of the input words, and then predict the masked words (Usually $k = 15\%$). Example :

I like going to the [MASK] in the evening

↓
park

- Too little masking: Too expensive to train
- Too much masking: Not enough context
- The model needs to predict 15% of the words, but we don't replace with [MASK] 100% of the time. Instead:
 - 80% of the time, **replace with [MASK]**
 - Example : like going to the park → like going to the [MASK]
 - 10% of the time, **replace random word**
 - Example : like going to the park → like going to the store
 - 10% of the time, **keep same**
 - Example : like going to the park → like going to the park









Next Sentence Prediction

- To learn relationships between sentences, predict whether Sentence B is actual sentence that proceeds Sentence A, or a random sentence.

Input = [CLS] I enjoy read [MASK] book ##s [SEP]
I finish ##ed a [MASK] novel [SEP]
Label = IsNext

Input = [CLS] I enjoy read ##ing book [MASK] [SEP]
The dog ran [MASK] the street [SEP]
Label = NotNext

- Important for many important downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI)
- How to choose sentences A and B for pretraining?
 - 50% of the time B is the actual next sentence that follows A (labeled as IsNext)
 - 50% of the time it is a random sentence from the corpus (labeled as NotNext)

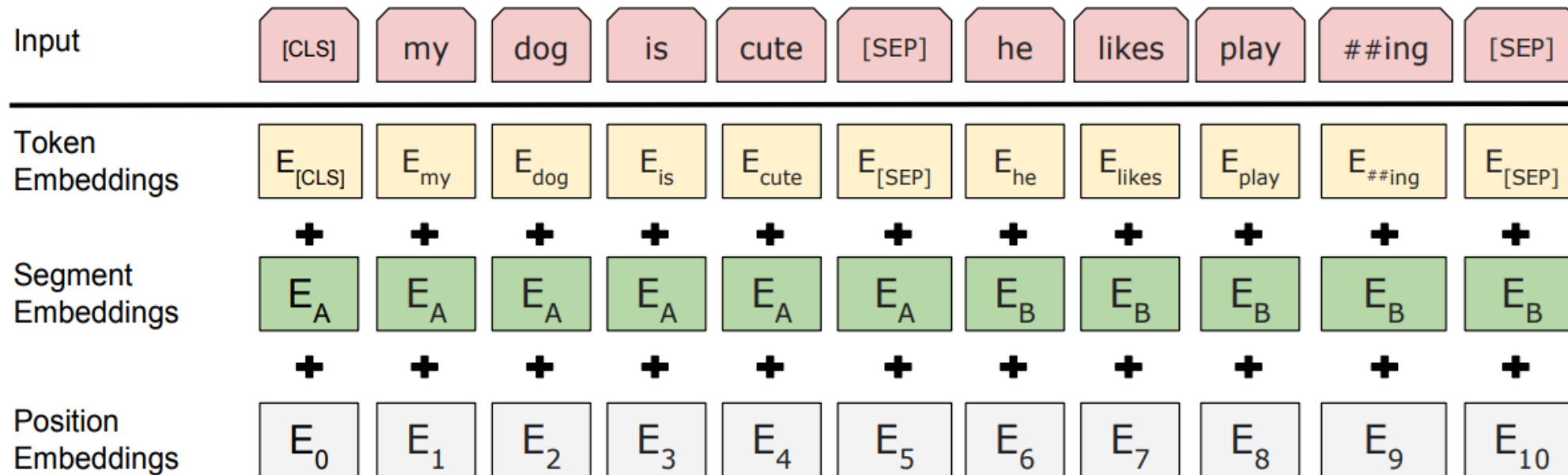






Input Representation

- Use 30,000 WordPiece vocabulary on input.
- For a given token, its input representation is constructed by summing the token embeddings, the segmentation embeddings and the position embeddings.



Source of Image : BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (Devlin et al., NAACL 2019)

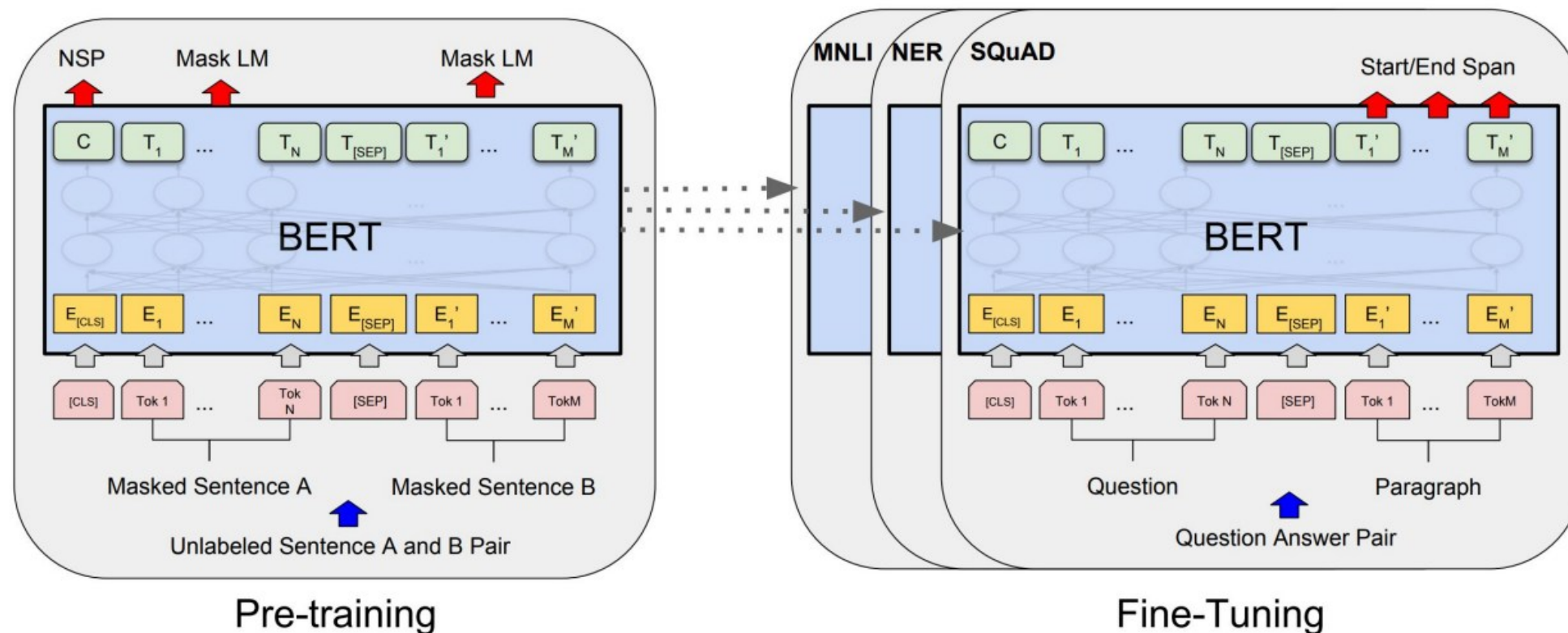


Training Details

- Data: Wikipedia (2.5B words) + BookCorpus (800M words)
- Batch Size: 131,072 words (1024 sequences * 128 length or 256 sequences * 512 length)
- Training Time: 1M steps (~40 epochs)
- Optimizer: AdamW, 1e-4 learning rate, linear decay
- BERT-Base: 12-layer, 768-hidden, 12-head
- BERT-Large: 24-layer, 1024-hidden, 16-head
- Trained on 4x4 or 8x8 TPU slice for 4 days



Fine-Tuning Procedure

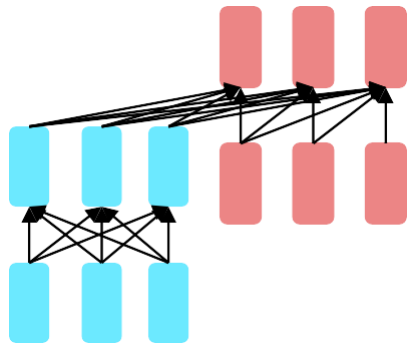






Pretraining for Three Types of Architectures

The neural architecture influences the type of pretraining, and natural use cases.



**Encoder-
Decoders**

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Pre-Training Encoder-Decoder Models

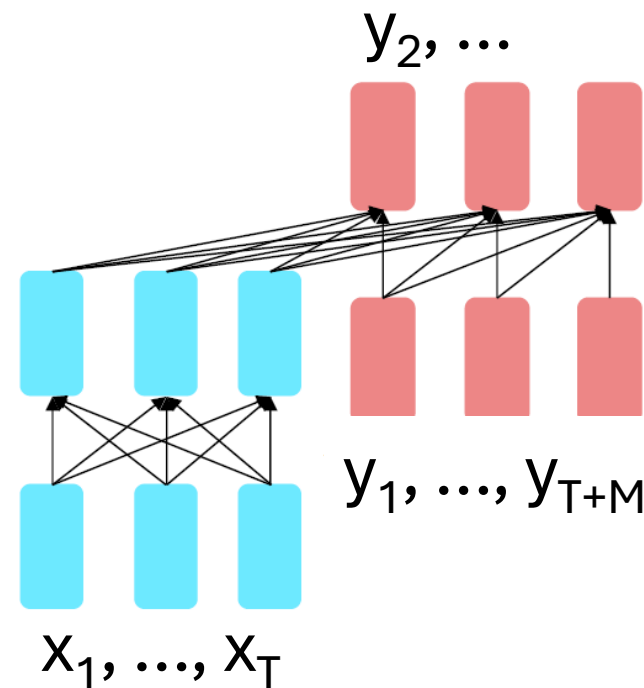
- For **encoder-decoders**, we could do something like **language modeling**, but where a prefix of every input is provided to the encoder and is not predicted.

$$h_1, \dots, h_T = \text{Encoder}(x_1, \dots, x_T)$$

$$h_{T+1}, \dots, h_{T+M} = \text{Decoder}(y_1, \dots, y_{i-1}, h_1, \dots, h_T)$$

$$P(y_i | y_{<i}, h_{1:T}) = \text{Softmax}(Wh_i + b)$$

The **encoder** portion benefits from bidirectional context; the **decoder** portion is used to train the whole model through language modeling.

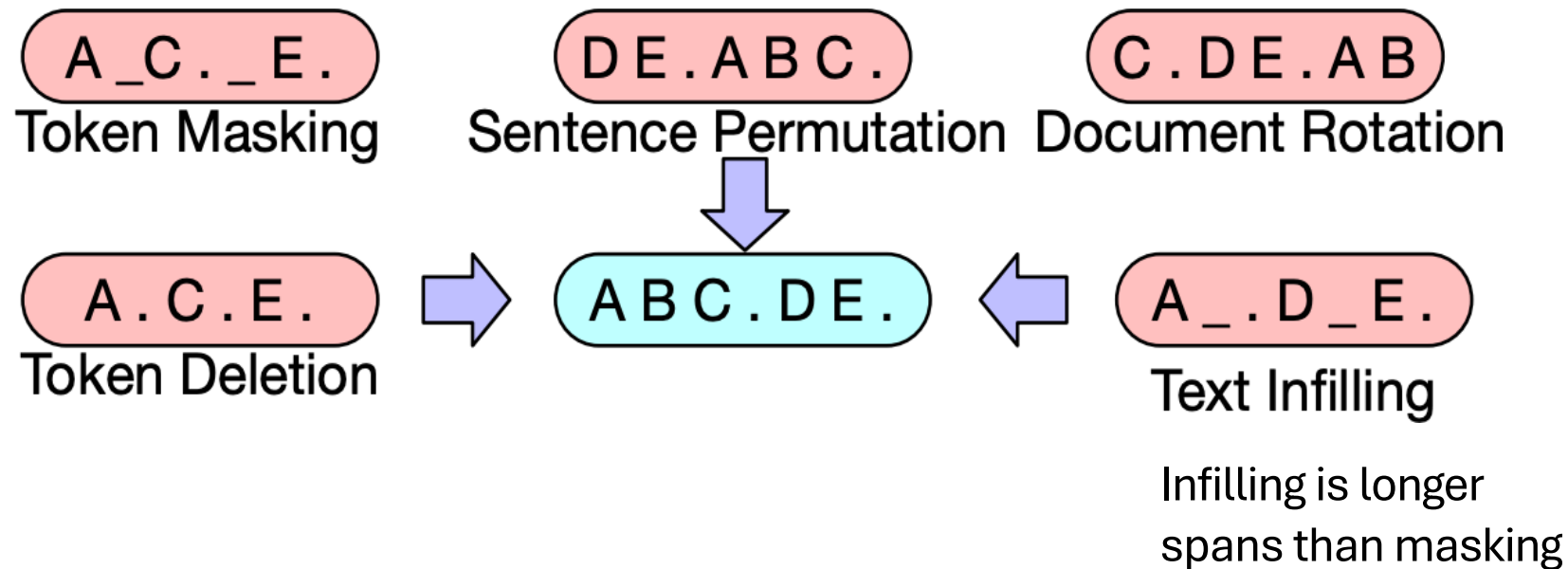


Pre-Training Encoder-Decoder Models

- How can we pre-train a model for $P(\mathbf{y} \mid \mathbf{x})$?
- **Requirements:**
 1. should use unlabeled data
 2. should force a model to attend from $\mathbf{y}$ back to $\mathbf{x}$



Pre-Training BART (Bidirectional and Auto-Regressive Transformers)



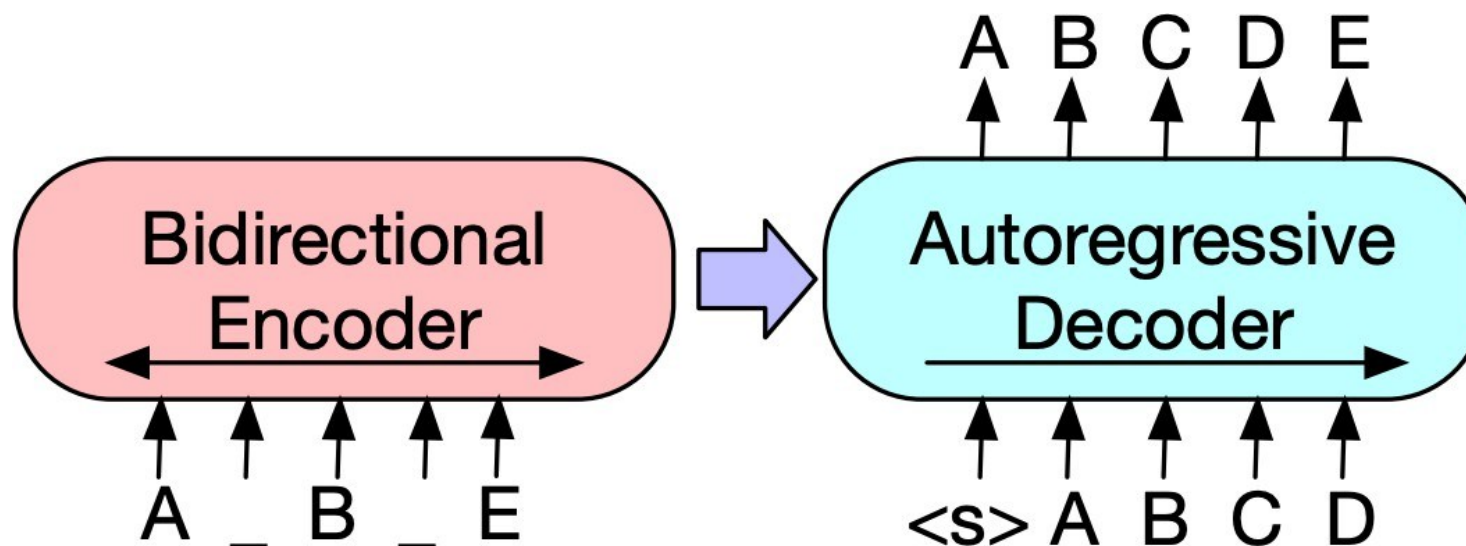
- Several possible strategies for corrupting a sequence are explored in the BART paper.

Lewis et al. (2019), "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension"



Pre-Training BART

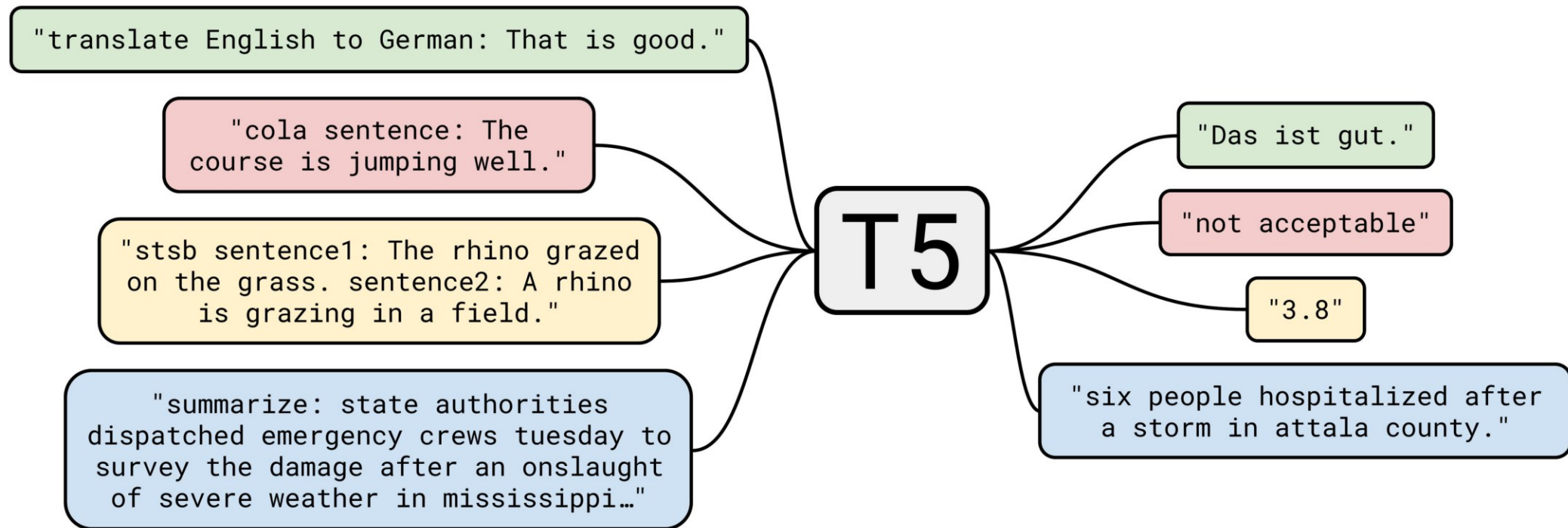
- Sequence-to-sequence Transformer trained on this data: permute/make/delete tokens, then predict full sequence autoregressively.



Lewis et al. (2019), "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension"



T5: Text-to-Text Transfer Transformer



Raffel et al. (2019), "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer"



Pre-Training T5

- **Pre-training:** similar denoising scheme to BART (they were released within a week of each other in fall 2019)
- **Input:** text with gaps ; **Output:** a series of phrases to fill those gaps.

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!

Raffel et al. (2019)



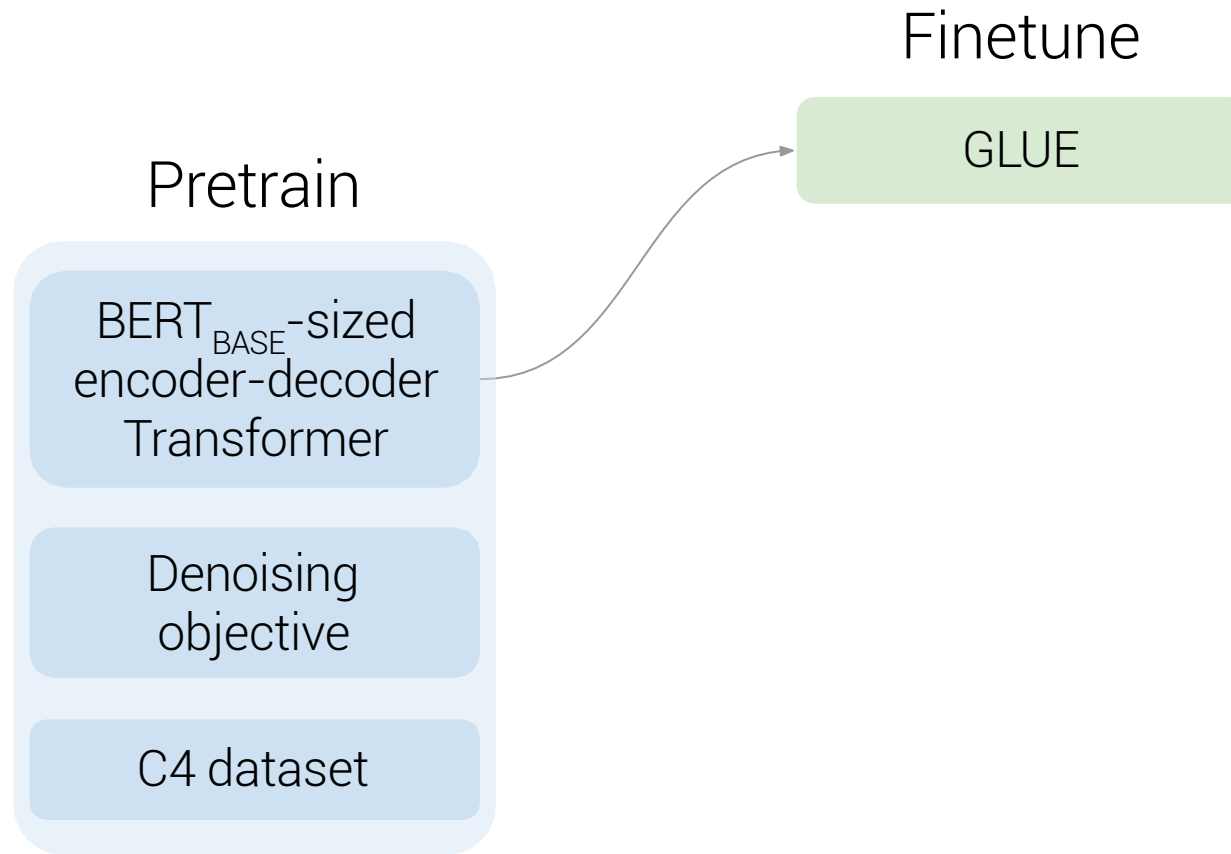
Pretrain

BERT_{BASE}-sized
encoder-decoder
Transformer

Denoising
objective

C4 dataset

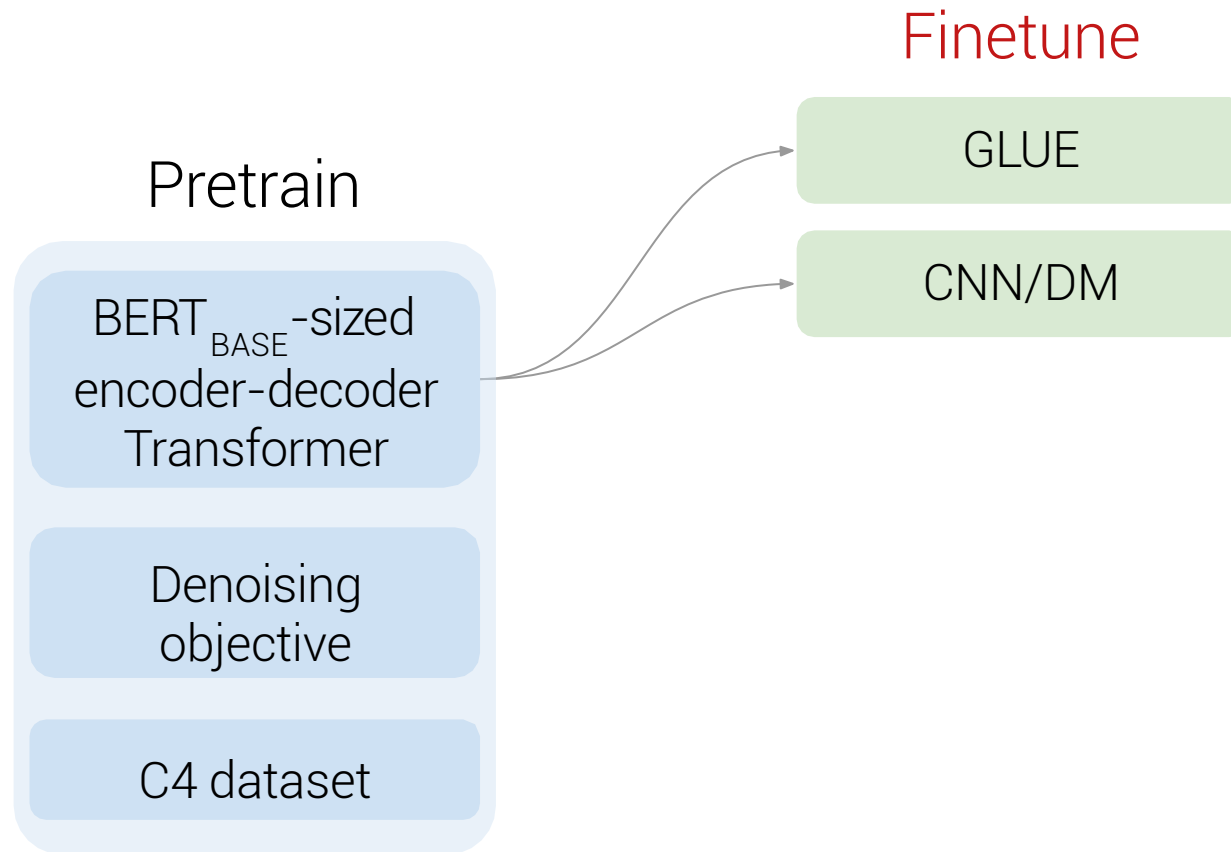


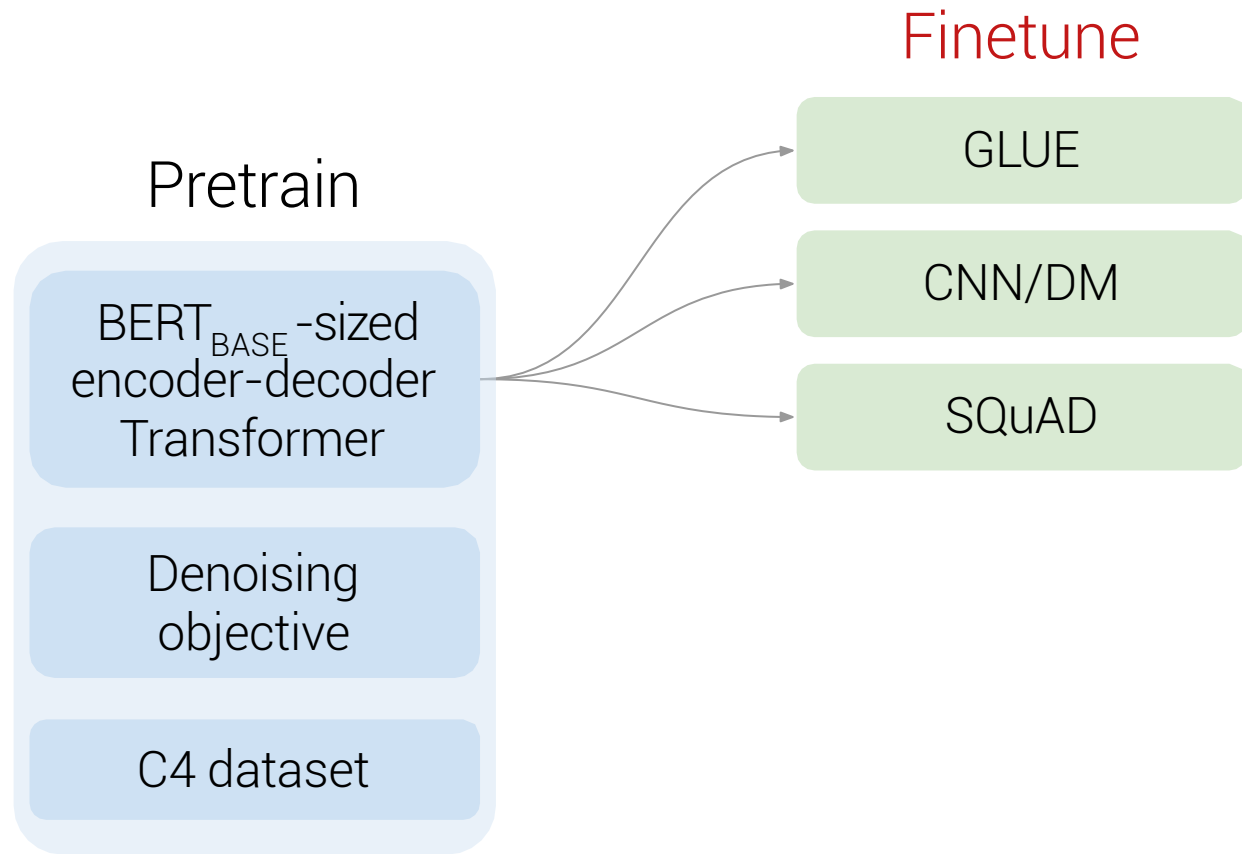


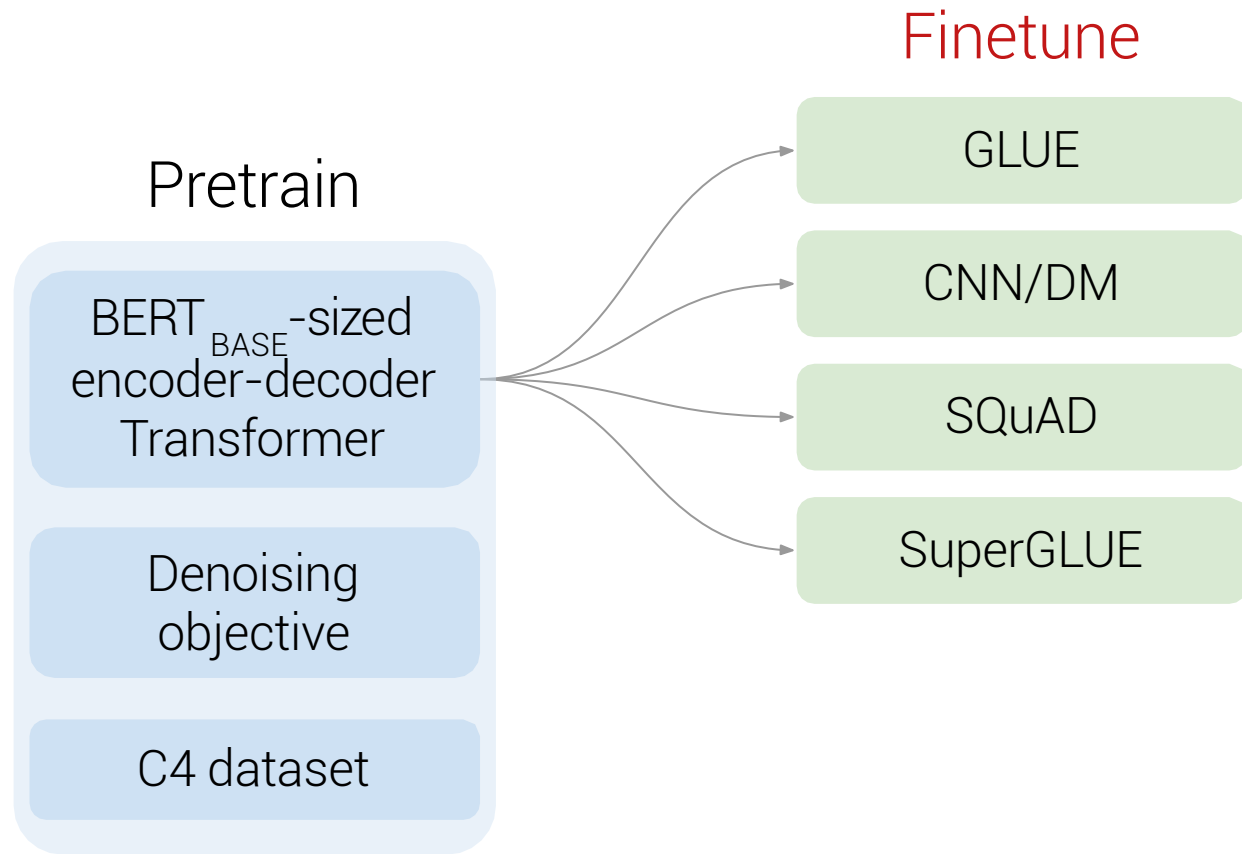
GLUE Benchmark

Dataset	Description	Data example	Metric
CoLA	Is the sentence grammatical or ungrammatical?	"This building is than that one." = Ungrammatical	Matthews
SST-2	Is the movie review positive, negative, or neutral?	"The movie is funny , smart , visually inventive , and most of all , alive ." = .93056 (Very Positive)	Accuracy
MRPC	Is the sentence B a paraphrase of sentence A?	A) "Yesterday , Taiwan reported 35 new infections , bringing the total number of cases to 418 ." B) "The island reported another 35 probable cases yesterday , taking its total to 418 ." = A Paraphrase	Accuracy / F1
STS-B	How similar are sentences A and B?	A) "Elephants are walking down a trail." B) "A herd of elephants are walking along a trail." = 4.6 (Very Similar)	Pearson / Spearman
QQP	Are the two questions similar?	A) "How can I increase the speed of my internet connection while using a VPN?" B) "How can Internet speed be increased by hacking through DNS?" = Not Similar	Accuracy / F1
MNLI-mm	Does sentence A entail or contradict sentence B?	A) "Tourist Information offices can be very helpful." B) "Tourist Information offices are never of any help." = Contradiction	Accuracy
QNLI	Does sentence B contain the answer to the question in sentence A?	A) "What is essential for the mating of the elements that create radio waves?" B) "Antennas are required by any radio receiver or transmitter to couple its electrical connection to the electromagnetic field." = Answerable	Accuracy
RTE	Does sentence A entail sentence B?	A) "In 2003, Yunus brought the microcredit revolution to the streets of Bangladesh to support more than 50,000 beggars, whom the Grameen Bank respectfully calls Struggling Members." B) "Yunus supported more than 50,000 Struggling Members." = Entailed	Accuracy
WNLI	Sentence B replaces sentence A's ambiguous pronoun with one of the nouns - is this the correct noun?	A) "Lily spoke to Donna, breaking her concentration." B) "Lily spoke to Donna, breaking Lily's concentration." = Incorrect Referent	Accuracy



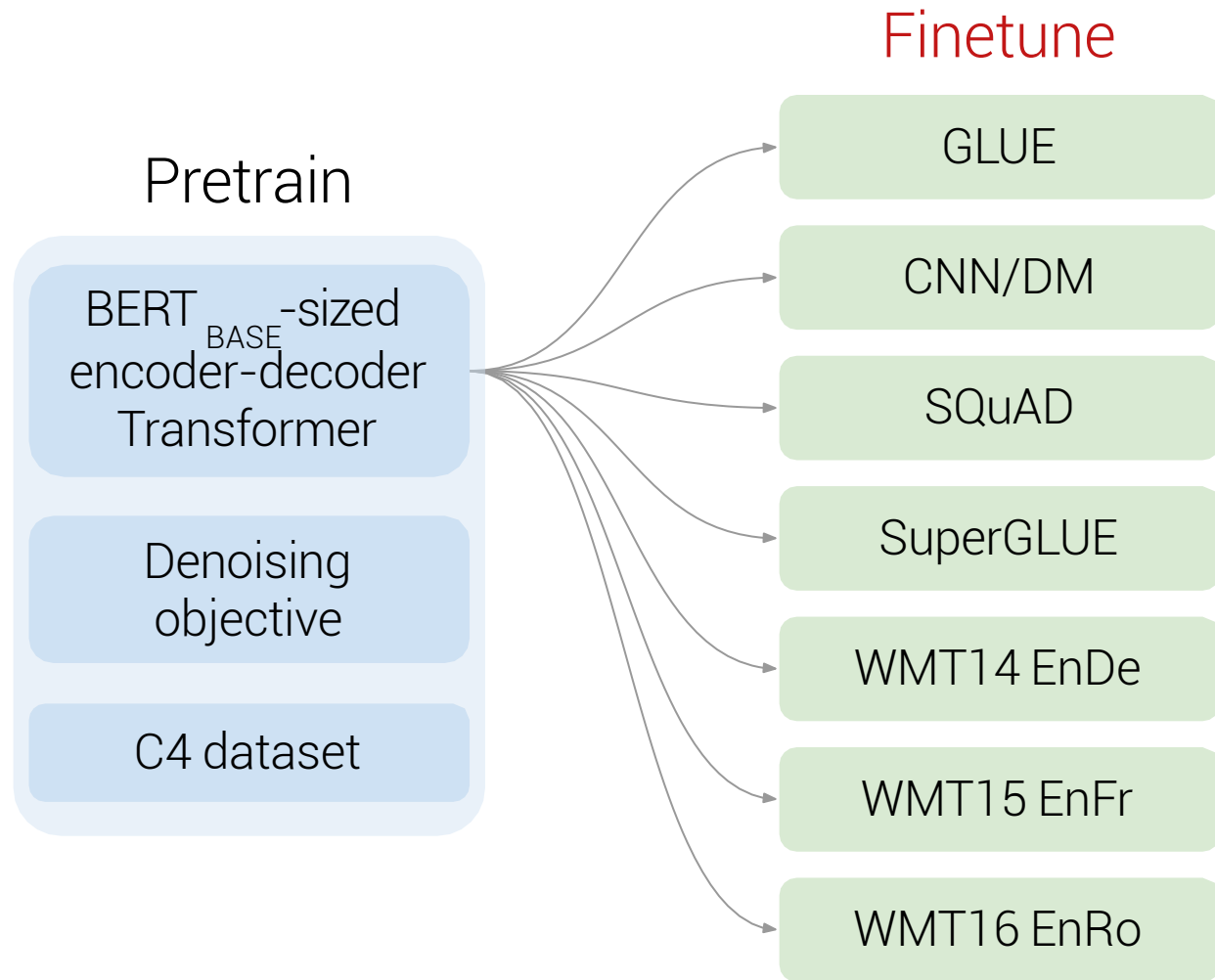


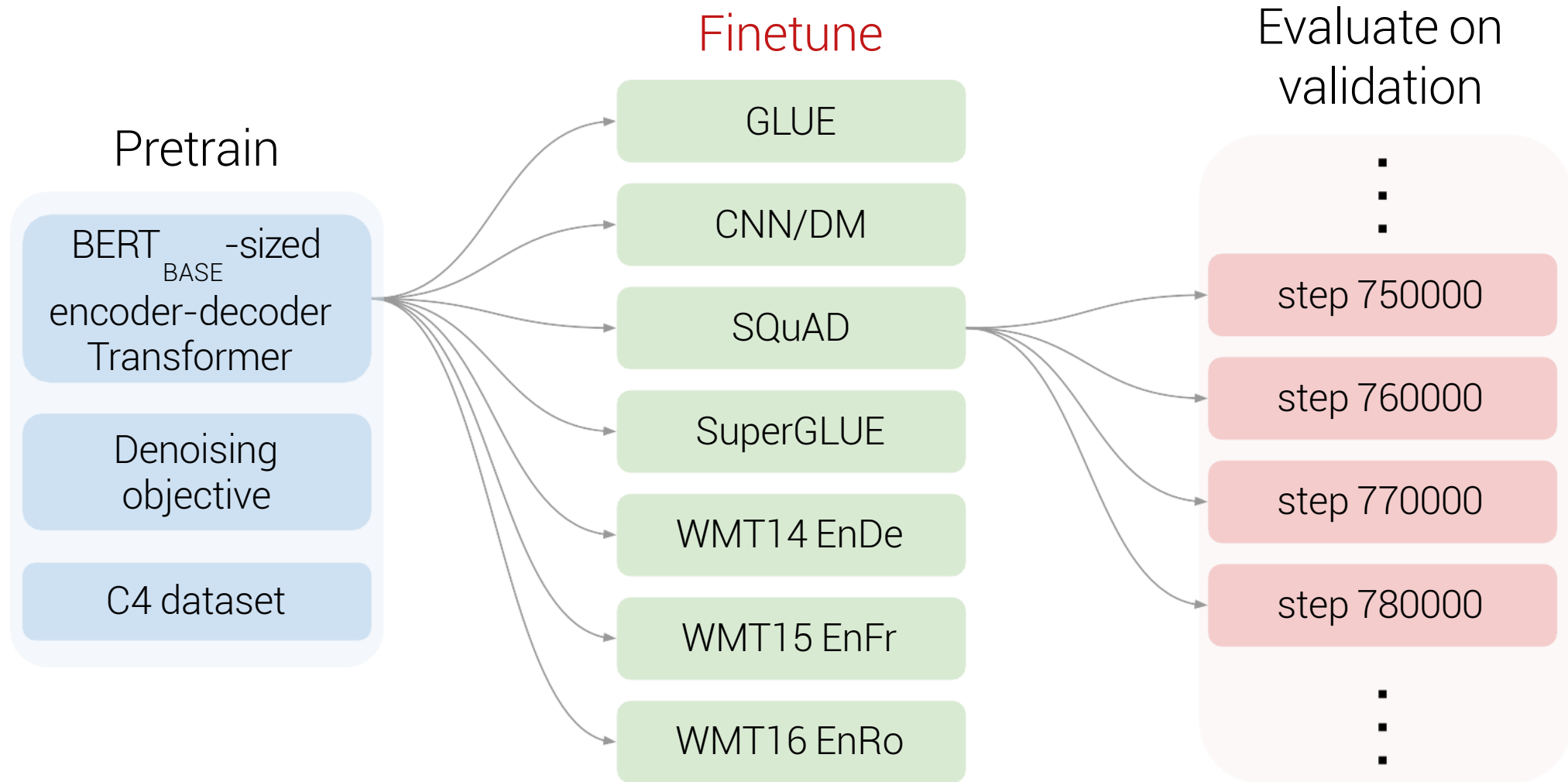




SuperGLUE Tasks

Name	Identifier	Download	More Info	Metric
Broadcoverage Diagnostics	AX-b			Matthew's Corr
CommitmentBank	CB			Avg. F1 / Accuracy
Choice of Plausible Alternatives	COPA			Accuracy
Multi-Sentence Reading Comprehension	MultiRC			F1a / EM
Recognizing Textual Entailment	RTE			Accuracy
Words in Context	WiC			Accuracy
The Winograd Schema Challenge	WSC			Accuracy
BoolQ	BoolQ			Accuracy
Reading Comprehension with Commonsense Reasoning	ReCoRD			F1 / Accuracy
Winogender Schema Diagnostics	AX-g			Gender Parity / Accuracy





Where Do the Pre-trained Models Fail?

Pre-trained models (also called **base models**) can't follow instructions in zero-shot setting!!

Example with Llama-3-8B-base [The first sentence is the input prompt]

[illegible]

Reason: Most of their training data is not in instruction-output format



How to make ChatGPT ?

- Pre-Training

- This is the point where most of the reasoning power is infused in the model.
- Data – Billions of tokens of unstructured text from the internet

- Instruction Tuning

- Trains models to follow natural language instructions
- Data – Several thousand (Task/Instruction, Output) examples

- Reinforcement Learning from Human Feedback

- Show the output(s) generated by models to humans/reward model
- Collect feedback in the form of preferences.
- Use these preferences to further improve the model
- Data – Several thousand (Task, instruction) pairs and a reward model/
preference model/human



But Instruction-tuning is Not Enough - Why?

- **Question:** What's the best way to lose weight quickly?

What to say?	What not to say?
Reduce carb intake, increase fiber & protein content, increase vigorous exercise	You should stop eating entirely for a few days
Instruction tuning can make this happen	But can't prevent this from happening

Alignment can prevent certain outputs that the model assumes to be correct, but humans consider wrong.



How to make ChatGPT ?

- Pre-Training

- This is the point where most of the reasoning power is infused in the model.
- Data – Billions of tokens of unstructured text from the internet

- Instruction Tuning

- Trains models to follow natural language instructions
- Data – Several thousand (Task/Instruction, Output) examples

- Reinforcement Learning from Human Feedback

- Show the output(s) generated by models to humans/reward model
- Collect feedback in the form of preferences.
- Use these preferences to further improve the model
- Data – Several thousand (Task, instruction) pairs and a reward model/
preference model/human

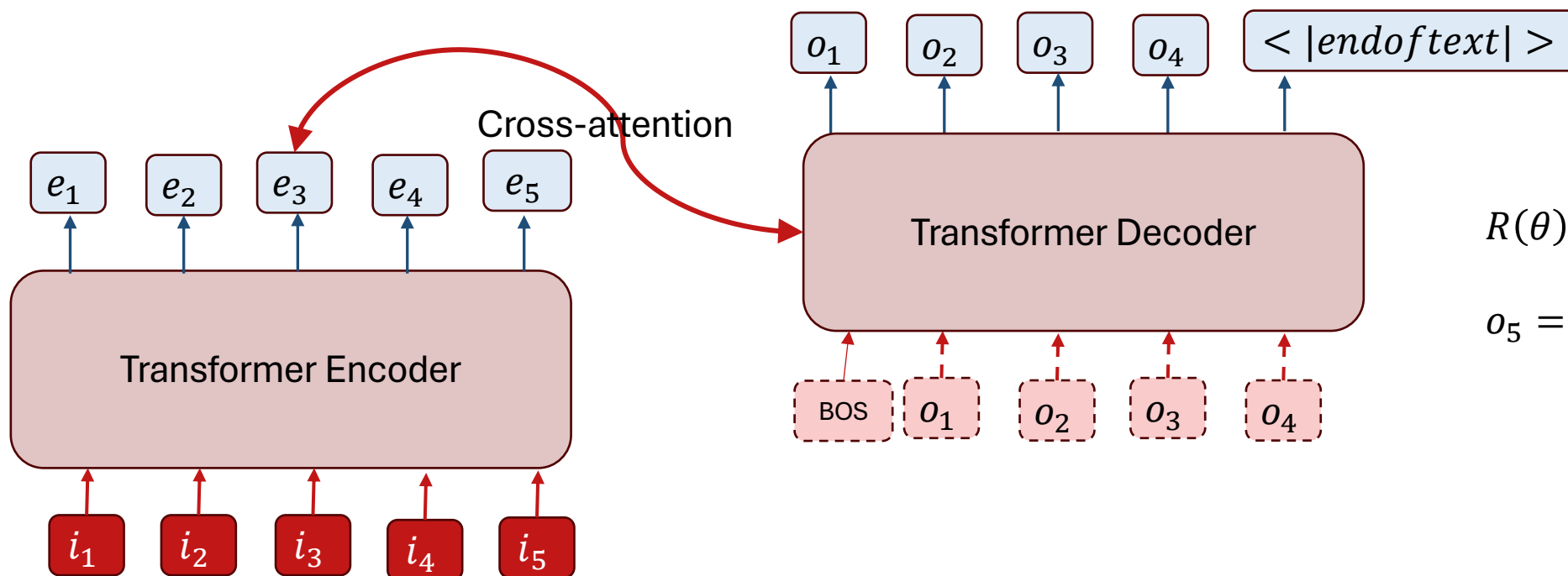


Training Loss



How to train? (Encoder-Decoder Models)

- Given (instruction, output) pairs
 - Tokenized *instruction* = $(i_1, \dots, i_m)$ *output* = $(o_1, \dots, o_n)$

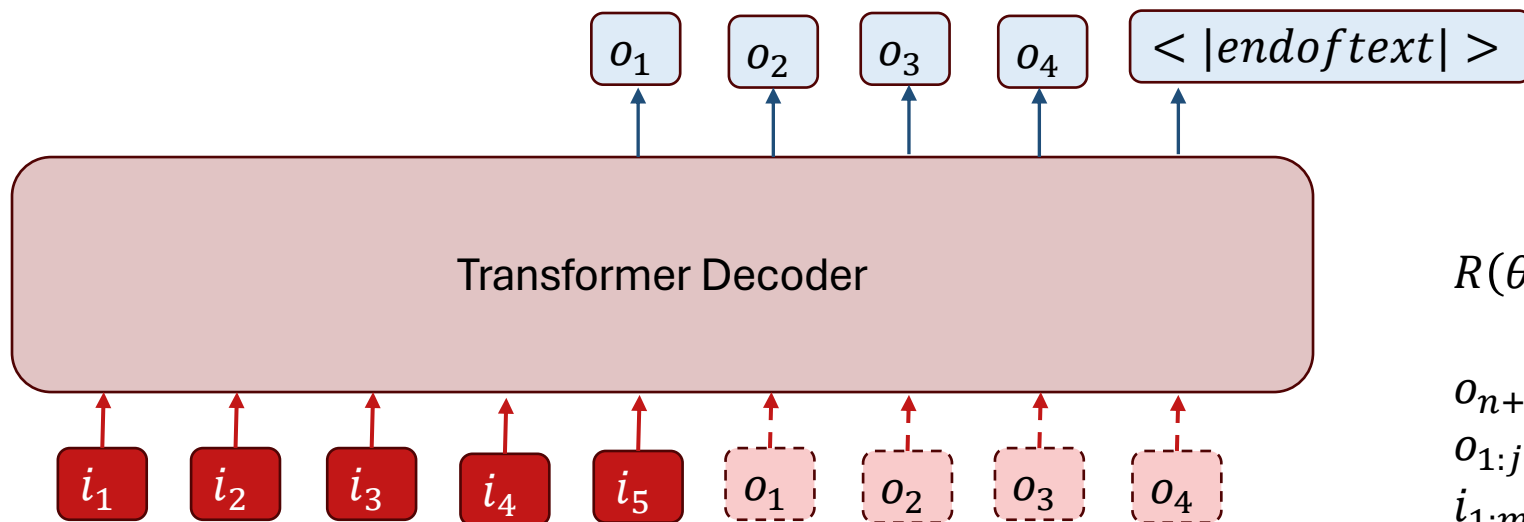


$$R(\theta) = \sum_{j=0}^n \log p_{\theta}(o_{j+1} | o_{1:j}, i_{1:m})$$
$$o_5 = < |endof\text{text}| >$$



How to train? (Decoder-only models)

- Given (instruction, output) pairs
 - Tokenized $instruction = (i_1, \dots, i_m)$ $output = (o_1, \dots, o_n)$



$$R(\theta) = \sum_{j=0}^n \log p_{\theta}(o_{j+1} | o_{1:j}, i_{1:m})$$

$$o_{n+1} = < |endof\text{text}| >$$

$$o_{1:j} = o_1, \dots, o_j$$

$$i_{1:m} = i_1, \dots, i_m$$



But is response-only loss optimal for decoder-only models?

- **WHY** zero-out loss on prompt tokens and backpropagate only on response tokens for decoder-only models – where both prompt and response are processed by the same decoder?
 - Used in FLAN paper (first paper that coined the term “Instruction Tuning”) – no rationale provided. Thereafter used widely till date – unquestioned!
- Seems to be a direct adaptation from SFT loss for classification tasks ...
- Isn't the conventional loss kind of like teaching a child how to given answers to questions but not teaching how to understand the questions themselves!
- So, is there more to it?



Weighted Instruction Tuning (WIT)

- Consider loss on both prompt and response tokens and weight them based on different factors (training data, model properties, downstream task, etc.)

$$\mathcal{L}_{\text{WIT}} = \frac{-1}{\sum_{i=1}^{N_{\mathcal{T}}} \left(\mathbb{I}(\lambda_p \neq 0) \cdot |\mathbf{P}_i| + \mathbb{I}(\lambda_r \neq 0) \cdot |\mathbf{R}_i| \right)} \times$$
$$\sum_{i=1}^{N_{\mathcal{T}}} \left[\lambda_p \sum_{j=1}^{|\mathbf{P}_i|} \log \mathbb{P}_{\mathcal{M}} \left(p_i^{(j)} \mid p_i^{(1)}, \dots, p_i^{(j-1)} \right) \right.$$
$$\left. + \lambda_r \sum_{j=1}^{|\mathbf{R}_i|} \log \mathbb{P}_{\mathcal{M}} \left(r_i^{(j)} \mid \mathbf{P}_i, r_i^{(1)}, \dots, r_i^{(j-1)} \right) \right] \quad (2)$$

TACL'25

On the Effect of Instruction Tuning Loss on Generalization

Anwoy Chatterjee^{*†}

Dept. of Electrical Engineering
Indian Institute of Technology Delhi
anwoy chatterjee@gmail.com

H S V N S Kowndinya Renduchintala[†]

Media and Data Science Research
Adobe Inc., India
rharisrikowndinya333@gmail.com

Sumit Bhatia

Media and Data Science Research
Adobe Inc., India
sumit.bhatia@adobe.com

Tanmoy Chakraborty

Dept. of Electrical Engineering
Indian Institute of Technology Delhi
tanchak@iitd.ac.in



Getting the Data



Where does the data come from?

- Human-crafted
 - Flan-2021
 - Transforms NLP benchmarks into natural language input-output pairs.

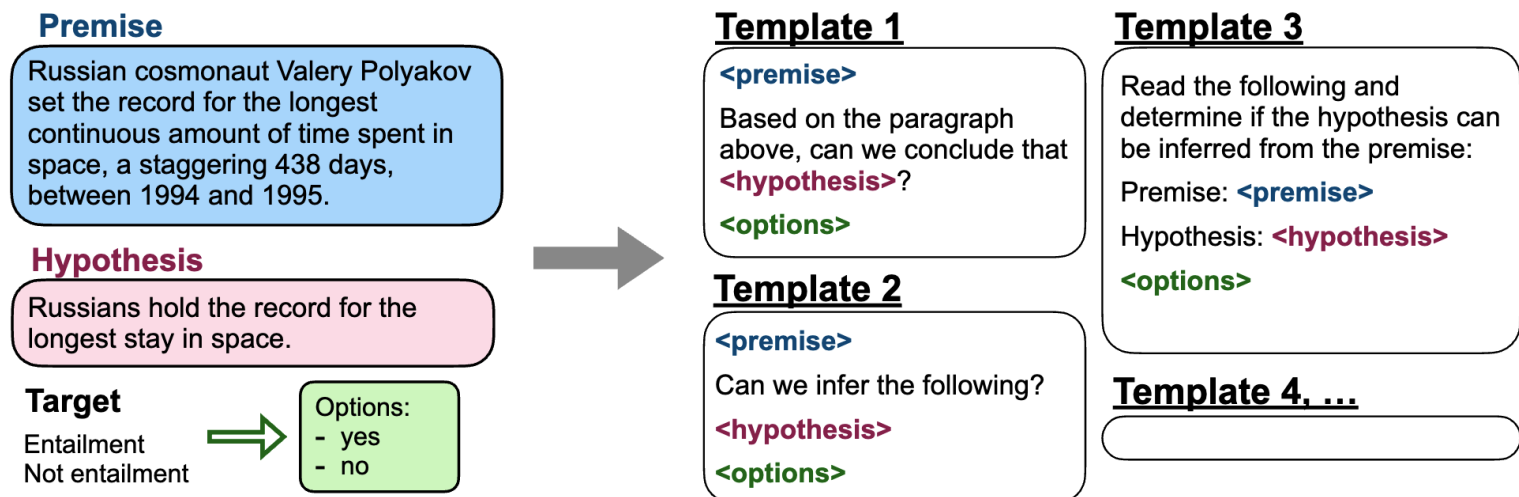


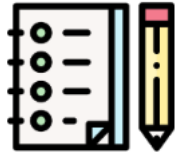
Figure 4: Multiple instruction templates describing a natural language inference task.

Credit: The Flan Collection: Designing Data and Methods for Effective Instruction Tuning



Self-Instruct

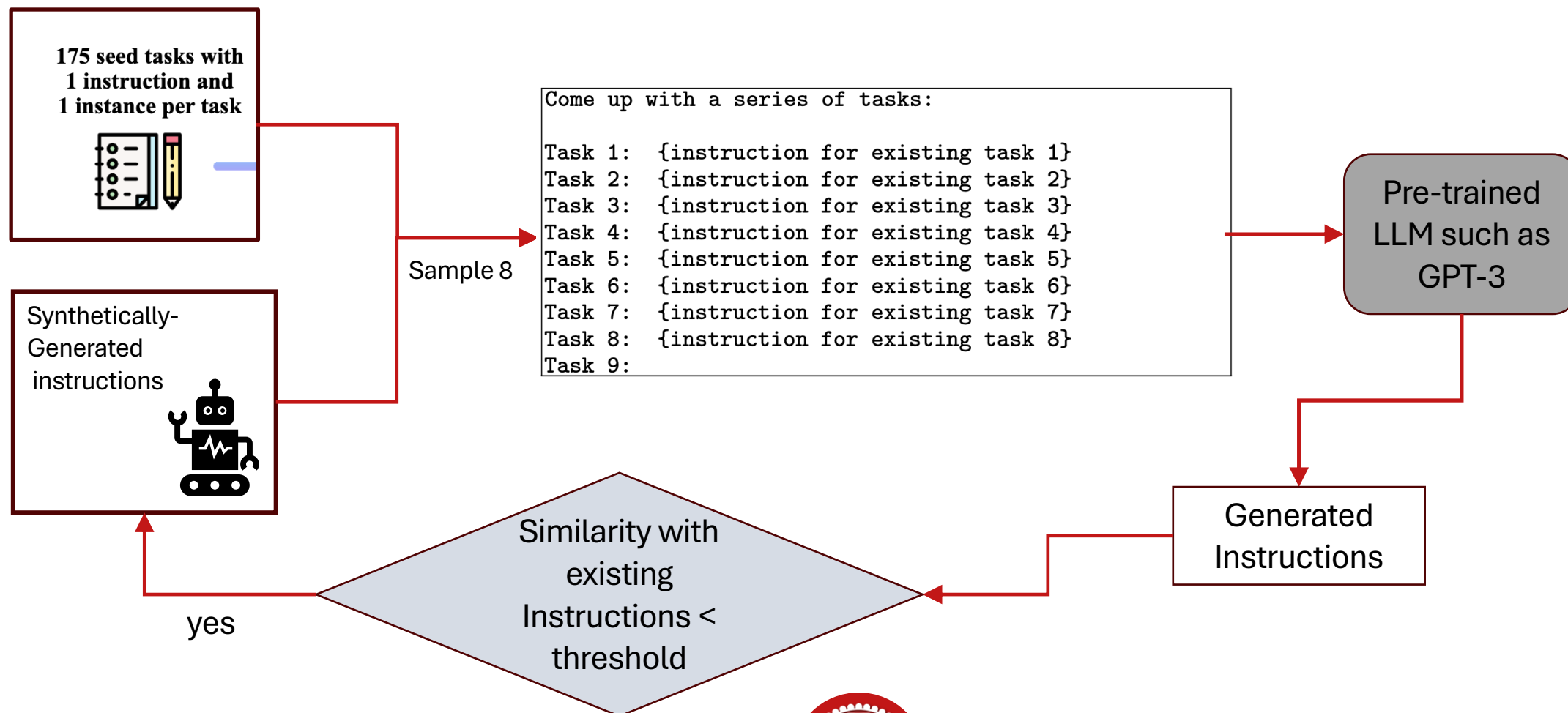
- Given: **175 seed tasks with
1 instruction and
1 instance per task**



- Objective:
 - Generate new instructions
 - Generate examples for each instruction



The Self-Instruct Process – Instruction Generation



Self-Instruct: The complete pipeline

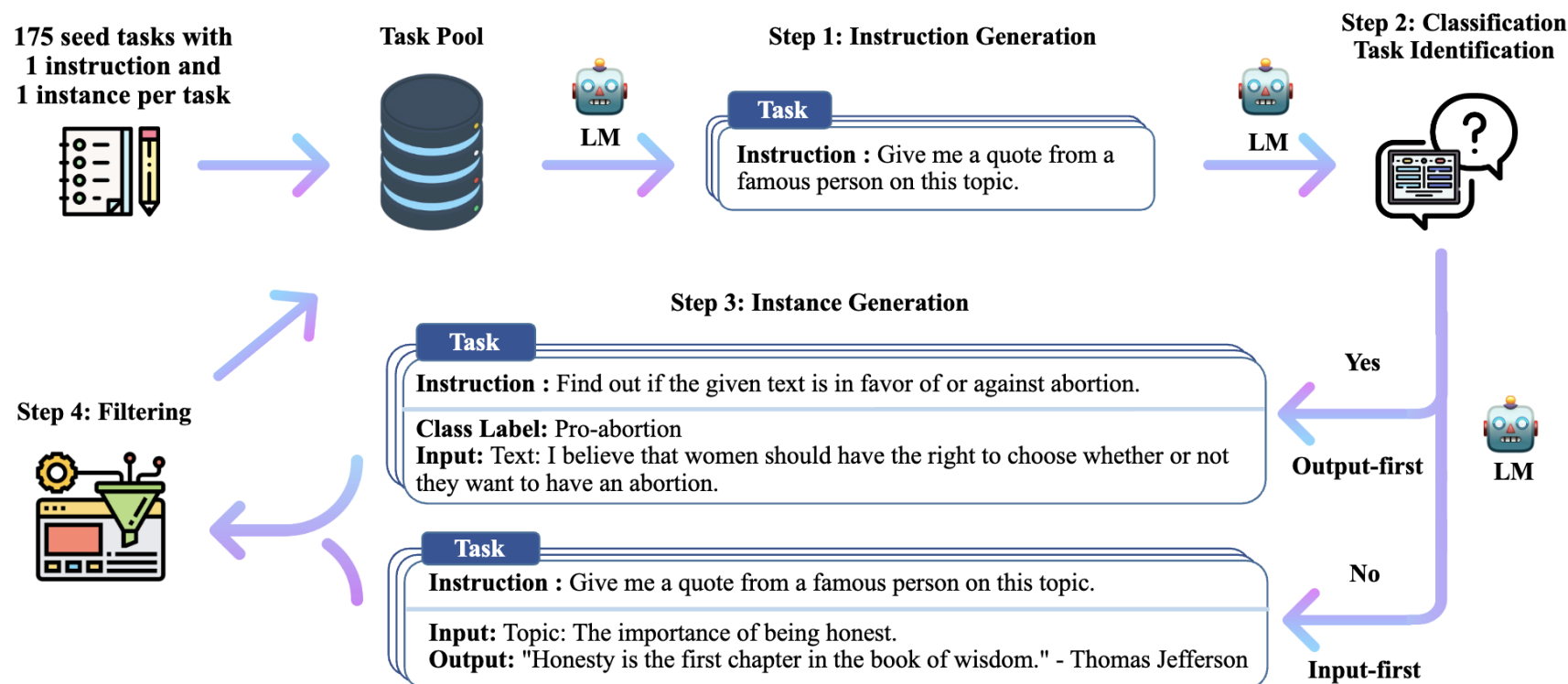
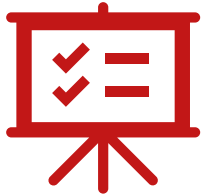


Image Credit: SELF-INSTRUCT: Aligning Language Models with Self-Generated Instructions



Evol-Instruct



Motivation:

Most of the instruction datasets contain only simple instructions.

LLMs can be used to make instructions more complex.



Instruction Evolver

An LLM that uses prompts to evolve instructions.



Instruction Eliminator

Checks whether the evolution fails.

- Non-informative responses



Instruction Evolver – In-Depth Evolution

- Add constraints
- Deepening
- Concretizing
- Increase Reasoning

I want you act as a Prompt Rewriter.

Your objective is to rewrite a given prompt into a more complex version to make those famous AI systems (e.g., ChatGPT and GPT4) a bit harder to handle.

But the rewritten prompt must be reasonable and must be understood and responded by humans.

...

You **SHOULD** complicate the given prompt using the following method: Please add one more constraints/requirements into **#Given Prompt#**

#Given Prompt#:

<Here is instruction.>

#Rewritten Prompt#:



Instruction Evolver – In-Breadth Evolution

- Enhance
 - Topic Coverage
 - Skill Coverage

I want you act as a Prompt Creator. Your goal is to draw inspiration from the #Given Prompt# to create a **brand new** prompt. This new prompt should belong to the same domain as the #Given Prompt# but be even more rare. The LENGTH and difficulty level of the #Created Prompt# should be similar to that of the #Given Prompt#.

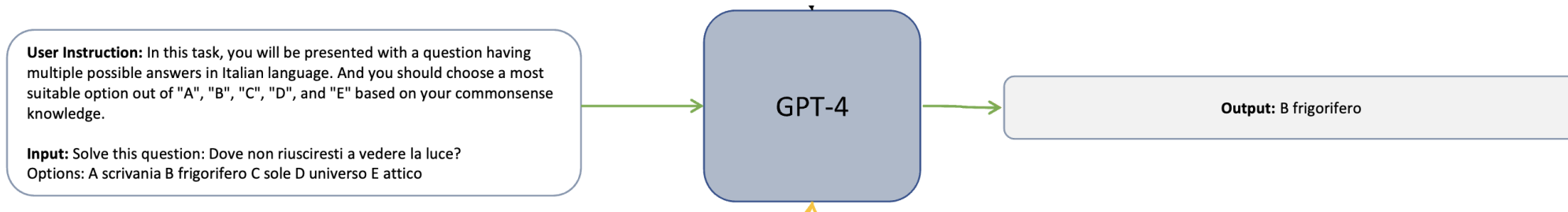
The #Created Prompt# must be reasonable and must be understood and responded by humans. ‘#Given Prompt#’, ‘#Created Prompt#’, ‘given prompt’ and ‘created prompt’ are not allowed to appear in #Created Prompt#.

#Given Prompt#:
<Here is instruction.>
#Created Prompt#:

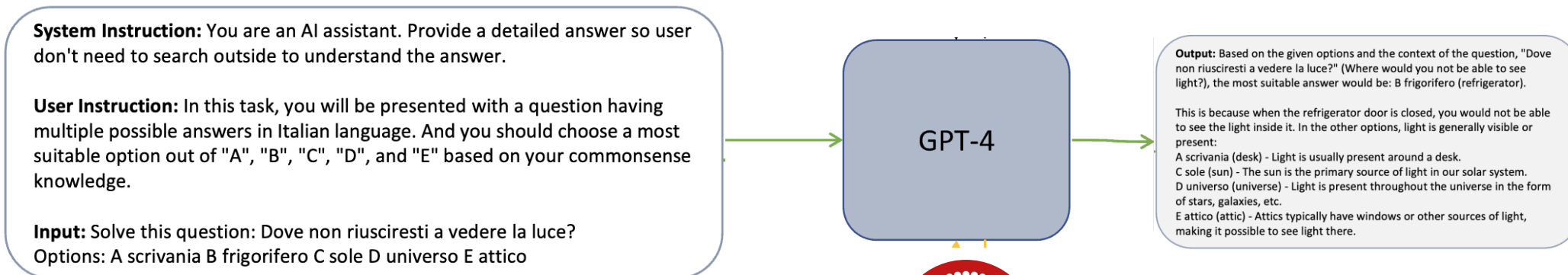


Orca

- How can we improve the information content in the response?



- Add a system instruction from a diverse instruction set including chain-of-thought, reasoning steps, explain like I'm five, being helpful and informative, etc.



Popular Instruction-Tuned Models on Known Datasets

- Flan-T5 (11B)
 - Fine-tuned T5-11B on **Flan** dataset
- Alpaca (7B)
 - Finetuned LLaMa-7B on synthetic dataset generated from text-davinci-003 generated using **Self-Instruct**
- WizardLM (7B)
 - Finetuned LLaMa-7B on an instruction dataset generated from ChatGPT using **Evol-Instruct**.
- Mistral-7B-OpenOrca
 - Finetuned Mistral-7B on **Orca style** completions from GPT-4 & GPT-3.5



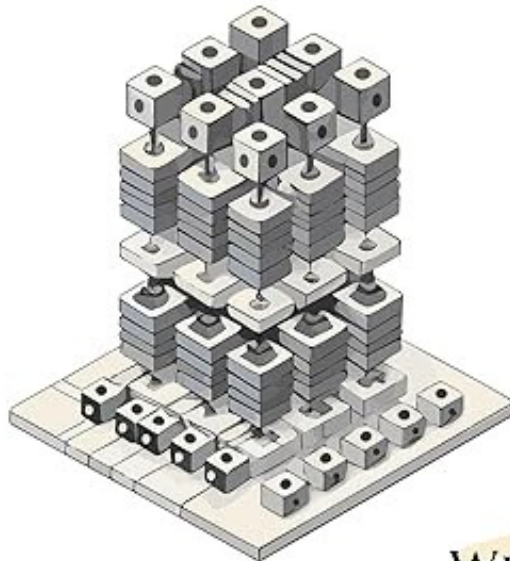
Parameter-Efficient Fine-tuning (PEFT)



INTRODUCTION TO LARGE LANGUAGE MODELS

Generative AI for Text

Tanmoy Chakraborty



WILEY

Chapter 10

Intro to LLMs – 2024

24



LLMs | Parameter Efficient Fine-Tuning (PEFT) | Lec 14.1

LCS2

25



LLMs | Quantization, Pruning & Distillation | Lec 14.2

LCS2

Adv LLMs – 2025



Lec 17 | Parameter-Efficient Fine-Tuning (PEFT)

1K views • 9 months ago



Lec 18 | Model Compression

541 views • 9 months ago



Lec 19 | Knowledge Distillation

545 views • 9 months ago



Challenges of Full Fine-Tuning

Memory and Computation Challenges

- Billions of parameters → GPU memory bottlenecks
- Training time is extensive
- Requires storing a full model copy for every task

Deployment Challenges

- Inflexible: Cannot easily switch between tasks
- Increases cost and carbon footprint

Motivation for Alternatives

Need more:

- Efficiency
- Scalability
- Modularity



In-Context Learning (ICL)

Idea - Feed it input-target examples (“shots”)

Example of a 4-shot input

Please unscramble the letters into a word, and write that word:

- asinoc=casino
- yfrogg=froggy
- plesim=simple
- iggestb=biggest
- astedro=?

Advantages

- No training required
- Fast and flexible

Disadvantages

- Prompt tokens consume context window
- Does not generalize or adapt across sessions
- Cannot learn from errors or improve performance



Parameter-Efficient Fine-Tuning (PEFT)

What is PEFT?

- A set of strategies to adapt large models by training only a small subset of parameters
- Core model remains frozen; auxiliary or selective parameters are updated

Motivations

- Reduce compute and memory footprint
- Allow reusability and modular adaptation
- Enable personalization and domain-specific training on edge devices

Three Main Categories

- **Additive Methods:** Add new modules (e.g., adapters, prompts)
- **Selective Methods:** Selectively update a sub-set of existing parameters
- **Re-parameterization Methods:** Re-express parameters to enable efficient tuning



Additive Methods - Adapters

Core Idea

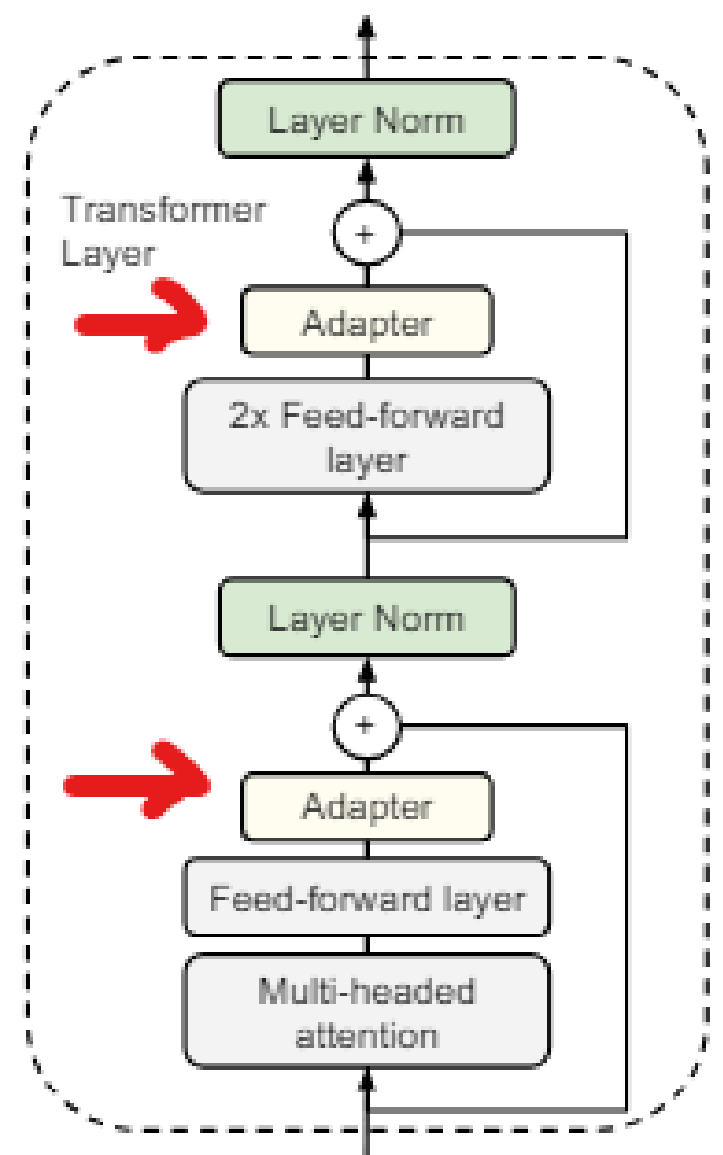
- Insert adapter layers in between the layers of transformer architecture
- Only adapters are updated; base model remains frozen

Advantages

- Now we can train and plug-in task specific adapters
- Small memory needed to save many task specific adapters

Disadvantages

- Increase in inference time due to additional layers
- Needs code changes to integrate into model architecture



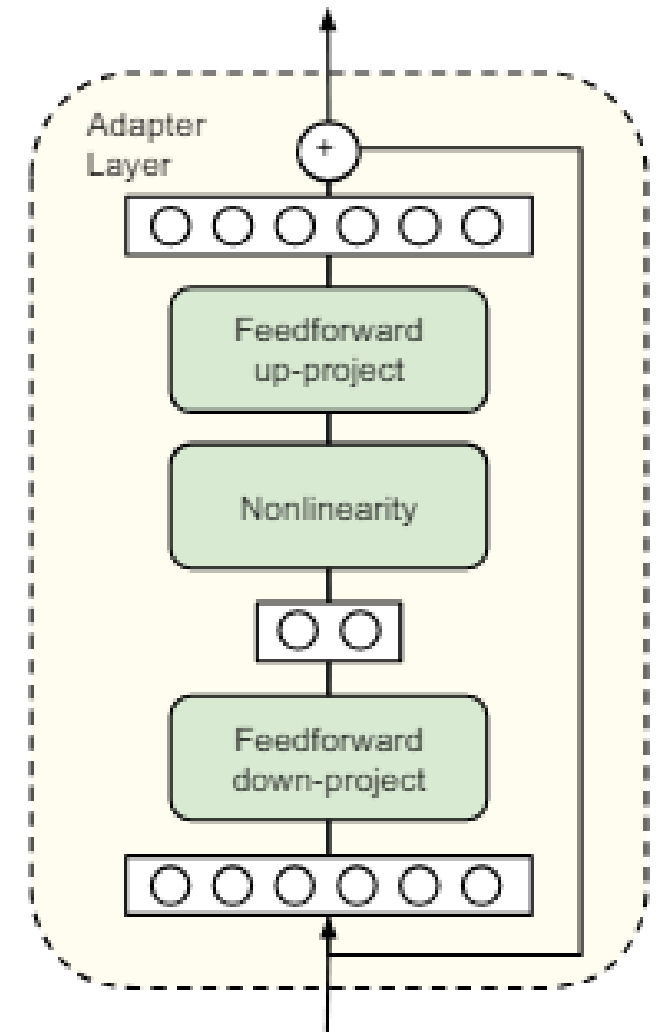
Additive Methods - Adapters

The Adapter Layer – Bottleneck structure

- Each adapter layer consists of one downward projection matrix, followed by a non-linear activation function and finally an upward projection
- There is also a residual connection

Results

- In **BERT**_{large}, with 3.6% additional parameters (adapters), able to achieve 99.6% performance of full-finetuning



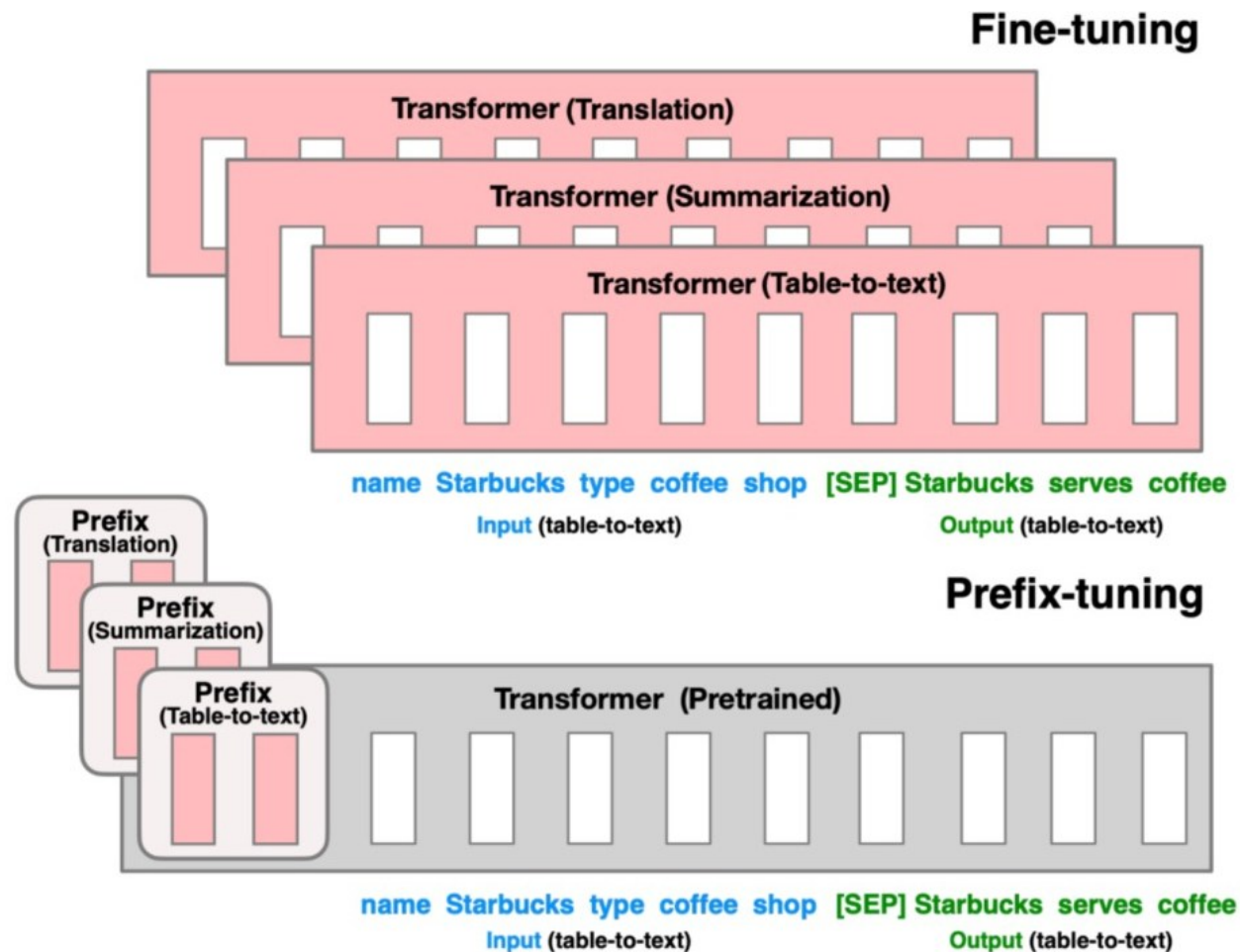
Additive Methods – Prefix-Tuning

Core Idea:

- Learn task-specific prefix key/value vectors that are prepended to each layer's self-attention

Method:

Learnable vector P is called **Soft Prompt**
input X of length n
prefix P of length m
modified input $X' = [P; X]$
shape of $X' = (n+m) \times d$
where d is model's hidden size



Additive Methods – Prompt-Tuning

Core Idea:

- Learn a continuous prompt embedding prepended to the input

Method:

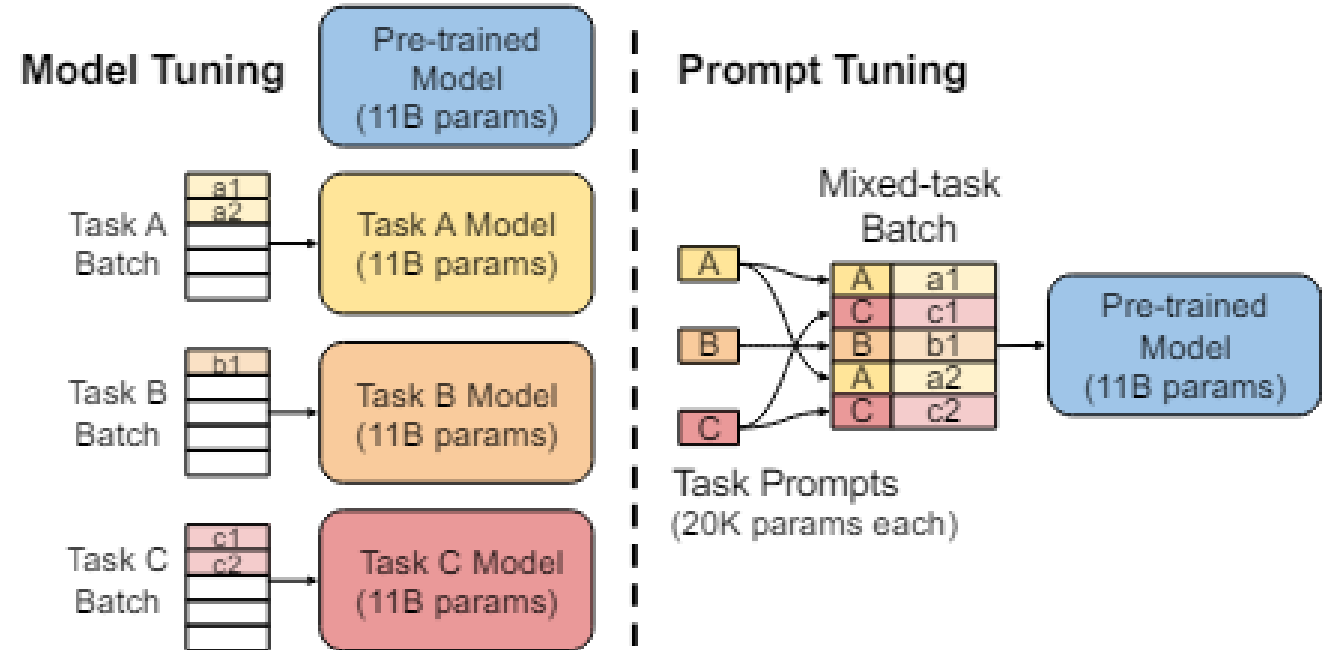
Input is: [Prompt Embeddings] + [Original Tokens]
Only prompt embeddings are updated

Advantages:

Multi-task input in a single batch (Training)
Very few parameters ($< 0.1\%$)

Disadvantages:

Less effective than prefix-tuning in low-data regimes



Selective Methods - Structured and Unstructured

Selective Methods:

- Selectively update a sub-set of existing parameters

Structured:

- Update specific components (e.g., attention weights only, FFN weights)
- Basically, update the entire tensor parameter (coarse update)

Unstructured:

- Update arbitrary subset of parameters (often selected by heuristics)
- A sub-set of scalar parameters in a tensor parameter (finer granularity)



Selective Methods - BitFit (Zaken et al.)

Idea:

- Only update bias terms of transformer layers

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell} \mathbf{x} + \mathbf{b}_q^{m,\ell}$$

$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell} \mathbf{x} + \mathbf{b}_k^{m,\ell}$$

$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell} \mathbf{x} + \mathbf{b}_v^{m,\ell}$$

BERT
encoder model

$$\mathbf{h}_2^\ell = \text{Dropout}(\mathbf{W}_{m_1}^\ell \cdot \mathbf{h}_1^\ell + \mathbf{b}_{m_1}^\ell) \quad (1)$$

$$\mathbf{h}_3^\ell = \mathbf{g}_{LN_1}^\ell \odot \frac{(\mathbf{h}_2^\ell + \mathbf{x}) - \mu}{\sigma} + \mathbf{b}_{LN_1}^\ell \quad (2)$$

$$\mathbf{h}_4^\ell = \text{GELU}(\mathbf{W}_{m_2}^\ell \cdot \mathbf{h}_3^\ell + \mathbf{b}_{m_2}^\ell) \quad (3)$$

$$\mathbf{h}_5^\ell = \text{Dropout}(\mathbf{W}_{m_3}^\ell \cdot \mathbf{h}_4^\ell + \mathbf{b}_{m_3}^\ell) \quad (4)$$

$$\text{out}^\ell = \mathbf{g}_{LN_2}^\ell \odot \frac{(\mathbf{h}_5^\ell + \mathbf{h}_3^\ell) - \mu}{\sigma} + \mathbf{b}_{LN_2}^\ell \quad (5)$$

- Bias terms are small in number but influential
- Extremely light-weight and easy to implement
- But it is architecture dependent (certain model architectures, e.g., Llama, do not have bias terms)



Selective Methods - PaFl (Liao et al.)

Idea:

- Select parameters based on magnitude

Steps:

1. Rank parameters based on magnitude
2. Select smallest - K% as trainable
3. Keep rest frozen

Advantages:

- Fixed masks
- Mask generation based on only magnitude (less computation cost)

Intuition

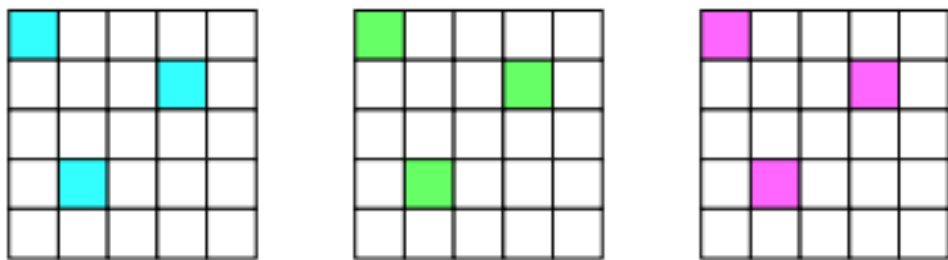
- Important parameters of a PLM learn a more generic representation (params with high magnitude)
- These params are important for downstream task as well
- Hence, keep these fixed and update low-magnitude params in fine-tuning
- We can argue that the important parameters of a PLM are also important for downstream task and needs to be updated. But authors empirically show that this is not the case.



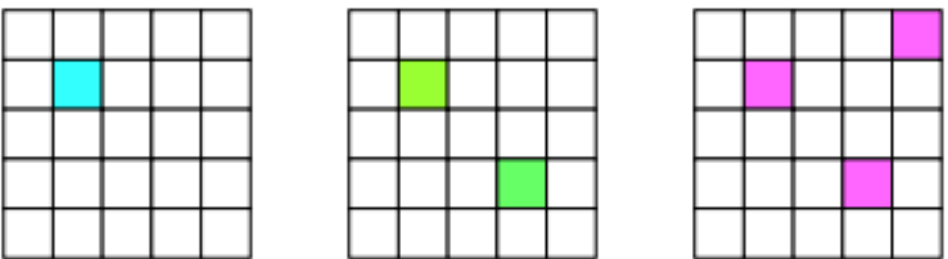
Selective Methods – ID3 (Agarwal et al., TACL'25)

Idea:

- Incremental selection of parameters instead of selecting all parameters in beginning
- Dynamic gradient & magnitude-based heuristic for selection of parameters in each optimizer step



Same subset of parameters trained in all steps



Incrementally select new parameters in each step

Given parameters θ^i and gradients ∇_{θ^i}

We define parameter importance function (also referred to as heuristic function) as:

$$\mathcal{H}(\theta^i) = \frac{|\nabla_{\theta^i}|}{(|\theta^i| + \epsilon)^{exp}}$$

Where ϵ and exp are hyperparameters to control smoothing of function and effect of parameter magnitude on final importance



Re-parameterization Methods

Idea

- Modify the form of parameters to allow efficient learning

Common trick

- Low-rank approximation: Replace weight W with $W + AB$

Popular Methods

- LoRA
- AdaLoRA
- DoRA
- QLoRA
- etc



Low-Rank Adaptation (LoRA)

Adapt a frozen pretrained model by learning a small low-rank update | Hu et al., ICLR 2022

Problem

Full fine-tuning updates every weight and carries optimizer state for each one -- and every downstream task needs its own full copy of the model.

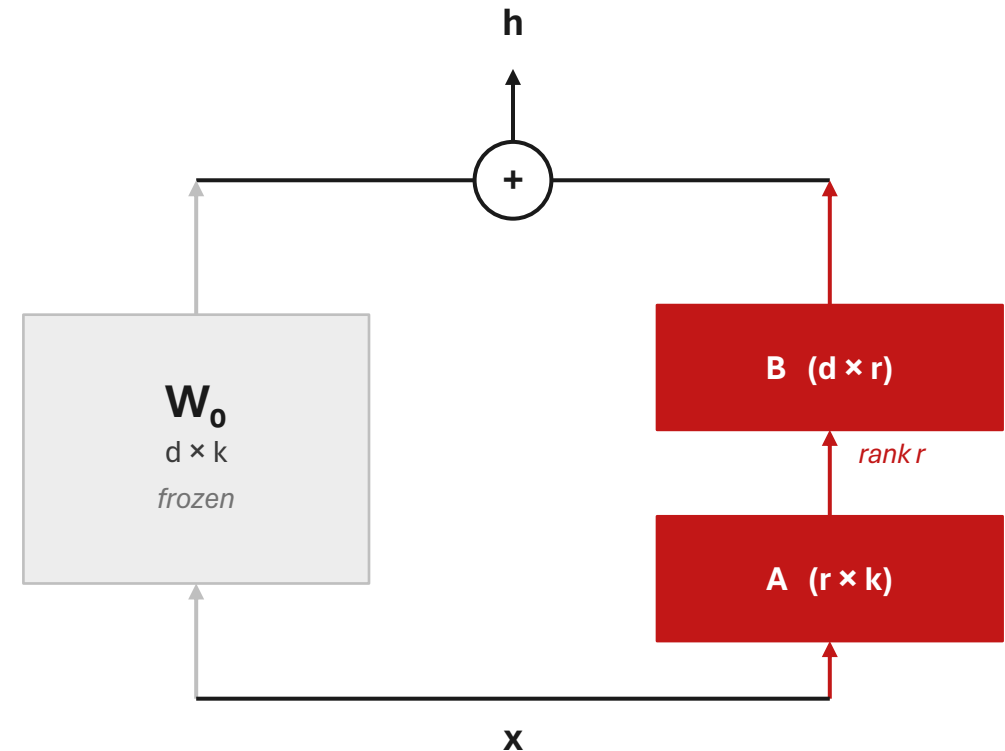
Hypothesis

The update ΔW that adapts a pretrained model has low intrinsic rank: a handful of directions account for almost all of it.

LORA

Freeze W_0 and learn $\Delta W = BA$ instead, where A has shape $r \times k$, B has shape $d \times r$, and $r \ll \min(d, k)$. Only A and B are trained.

$$h = W_0 x + \Delta W x = W_0 x + (\alpha / r) \cdot B A x$$



Gradients flow only through A and B . W_0 is frozen and shared across every task.



Anatomy of a LoRA adapter

What you actually pay for, and the four knobs that decide the outcome

0.4%

trainable parameters for one 4096×4096 projection at $r = 8$ — 65K instead of 16.8M

3×

less GPU memory than full fine-tuning: no optimizer state for the frozen backbone

0

extra inference latency — BA merges back into W_0 , leaving the deployed model unchanged

The four knobs

1 Rank r

Capacity of the update. $r = 4-16$ covers most task adaptation; go higher for domain shift, new languages, or long instruction-tuning runs.

2 Scaling α

The branch output is scaled by α / r , so changing r does not change the effective step size. In practice $\alpha / r \approx 1-2$ works well.

3 Placement

The original paper adapts W_q and W_v only. Adapting all attention projections plus the MLP helps once you have plenty of data.

4 Initialisation

A is Gaussian, B is zero. So $\Delta W = 0$ at step 0 and training starts exactly at the pretrained model, with no warm-up shock.



Variants, limits, and defaults

- **QLoRA** 4-bit NF4 quantised frozen base + LoRA in bf16. Puts 65B-scale fine-tuning on a single 48 GB GPU.
- **DoRA** Decomposes ΔW into magnitude and direction; recovers much of the full fine-tuning gap at low rank.
- **AdaLoRA** Allocates rank budget across layers by SVD-based importance rather than fixing r everywhere.
- **LoRA+** Separate learning rates for A and B -- B needs the larger one for the update to be well-scaled.
- **rsLoRA** Scales by $\alpha / \sqrt{r}$ instead of α / r , which keeps high-rank training stable.
- **MonteCLoRA** Bayesian LoRA: mixture-of-Gaussians adapters estimated by Monte Carlo, cutting LoRA's hyperparameter sensitivity for $O(r)$ extra parameters. (TMLR 2025)

Defaults to start from

$r = 8-16$ · $\alpha = 2r$ · dropout= 0.05
target every linear layer, not just q and v
learning rate $1e-4$ to $3e-4$ -- roughly $10\times$ what full fine-tuning takes

